A Project Report on

RoomRover - Smart IoT Based Navigation Assistance

submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING

By

Maitreyi Sameer Phadke: 22106007

Gauri Kishor Ramekar: 22106068

Arya Surendra Raut: 22106075

Chinmay Ajit Sawant: 22106017

Under the Guidance of

Prof. Sayali Badhan

Department of Computer Science & Engineering

(Artificial Intelligence & Machine Learning)

A.P. SHAH INSTITUTE OF TECHNOLOGY, THANE

UNIVERSITY OF MUMBAI

2025-2026

A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

CERTIFICATE

This is to certify that the project entitled "RoomRover – Smart IoT Based Navigation Assistance" is a bonafide work of **Maitreyi Sameer Phadke(22106007), Gauri Kishor Ramekar(22106068), Arya Surendra Raut(22106075), Chinmay Ajit Sawant(22106017)** submitted to the University of Mumbai in partial fulfilment of the requirement for the award of the degree of **Bachelor of Engineering** in **Computer Science & Engineering (Artificial Intelligence & Machine Learning**).

Prof. Sayali Badhan
Guide

Prof. Sayali Badhan
Project Co-ordinator

Dr. Jaya Gupta
Head of Department

Dr. Uttam Kolekar
Principal

A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

Project Report Approval for B.E.

This project report entitled ***RoomRover-Smart Iot Based Navigation Assistance*** by ***Maitreyi Phadke, Gauri Ramekar, Arya Raut, Chinmay Sawant*** is approved for the degree of ***Bachelor of Engineering*** in ***Computer Science & Engineering (Artificial Intelligence & Machine Learning)***, ***2025-26***.

Examiner Name	Signature
1._____	_____
2._____	_____

Date:

Place:

Declaration

We declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Maitreyi Sameer Phadke 22106007

Gauri Kishor Ramekar 22106068

Arya Surendra Raut 22106075

Chinmay Ajit Sawant 22106017

Date:

Abstract

Indoor systems for monitoring usually decouple physical mobility and intelligent reasoning capabilities, which may confine robot platforms to just movement without contextual intelligence and interactive reasoning capabilities. This paper introduces RoomRover, a unifying autonomous system which combines contextual and smart IoT-based navigation assistance with intelligent cognitive assistance in one single system architecture. The system utilizes a multi-layered infrastructure that combines Internet of Things (IoT) with Retrieval Augmented Generation (RAG) techniques for contextual responses. At the hardware level, an ESP32-based smart mobile system relies on ultrasonic and MQ2 arrays for obstacle detection and environmental risk detection. Additionally, the system uses cloud-based telemetry services. At the cognitive-level implementation, a voice assistant is integrated based on the STT technique using the FAISS-based semantic retrieval module and a GPT-2-based transformer. Further, the system avoids the computationally heavy implementation of SLAM technology, ensuring a low-latency performance. Apart from the technical contributions of the system, RoomRover works towards safer and more intelligent indoor settings, assistive navigation, detection of hazards, and better accessibility. Knowledge of the system is important in the realization of the globally accepted sustainability requirements with safe communities, infrastructure, and technology-led societal well-being. Scalability of the system is validated with experimentation suitability in smart environments. Furthermore, the RoomRover project aligns with key United Nations Sustainable Development Goals (SDGs), including **Goal 3 (Good Health and Well-being)** by enhancing indoor safety through monitoring and hazard detection, **Goal 9 (Industry, Innovation and Infrastructure)** through the integration of IoT, robotics, and AI, and **Goal 11 (Sustainable Cities and Communities)** by enabling smart, safe, and resilient indoor environments, while remaining scalable, cost-effective, and adaptable for future improvements.

Keywords: IoT, Autonomous Robot, ESP32, Artificial Intelligence, Voice Assistant, Retrieval-Augmented Generation (RAG), Blynk Cloud, Obstacle Avoidance, Smoke Detection, Speech-to-Text, Smart Environment, Indoor Monitoring, Hazard Detection, Sustainable Technology

Contents

1. Introduction....2
1.1 Need for Smart Integrated Systems....3
1.2 Role of AI, IoT, and Emerging Technologies....3
1.3 Overview of Blynk IoT Platform....4
2. Survey....5
2.1 Research Gap Identification....13
3. Limitation of Existing System....14
4. Problem Statement and Objective....17
4.1 Problem Statement....17
4.2 Objective....18
4.3 Scope....19
5. Proposed System....20
6. Experimental Setup....23
6.1 Platform....23
6.2 Language....23
6.3 Build Tool....24
6.4 Software Requirements....24
6.5 Operating Environment....24
6.6 Development Environment....25
6.7 Hardware Requirements....25
6.8 Machine Learning & NLP Components....27
7. Results and Discussion....28
7.1 Physical Model of the system....28
7.2 Real-Time Navigation Execution....29
7.3 Destination Reached Confirmation....30
7.4 Fire Detection and Alert Generation....30
7.5 Smoke Monitoring with Fire Threshold....31
7.5 SOS Emergency Notification via Email....32
7.6 Chatbot User Interface....32
7.7 Performance Analysis....33
7.7.1 User Notification (Email Alert via SMTP)....33
7.7.2 Sensor Data Acquisition....34
7.7.3 Chatbot Interaction and Response Time....34

7.8 Overall System Performance Evaluation and Analysis 35
8. Conclusion and Future Work 37
8.1 Conclusion 37
8.2 Future Scope 38
References 39
Appendix 41
Publication 44
Acknowledgements 57

LIST OF FIGURES

Figure 5.1: Proposed System 20
Figure 6.7.1: Hardware Components of the system 25
Figure 7.1: Physical Model of RoomRover System 28
Figure 7.2: Robot Navigation in Progress 29
Figure 7.3: Successful Arrival at Destination 30
Figure 7.4: Fire Detection Using Smoke Sensor 30
Figure 7.5: Automated SOS Alert Notification 32
Figure 7.6: Chatbot Interface for User Interaction 32

LIST OF TABLES

Table 2.1: Research Gap 11
Table 7.7.1: End-to-End Delay (Email Notification via SMTP) 33
Table 7.7.2: End-to-End Delay (Sensor Data Acquisition) 34
Table 7.7.3: End-to-End Delay (Chatbot Response Time) 35

LIST OF GRAPHS

Graph 7.5: Smoke Monitoring with Fire Threshold (ppm) 31
Graph 7.8 System Efficiency Comparison Across Five Trials 35

ABBREVAION

AI	*Artificial Intelligence*
ML	*Machine Learning*
IOT	*Internet of Things*
SLAM	*Simultaneous Localization and Mapping*
ROS	*Robot Operating System*
FAISS	*Facebook AI Similarity Search*
BERT	*Bidirectional Encoder Representations from Transformers*
NLP	*Natural Language Processing*
RAG	*Retrieval-Augmented Generation*

Chapter 1

Introduction

1. Introduction

In the rapidly evolving landscape of smart technology, the integration of autonomous systems into daily environments has become a critical area of innovation. While standalone devices like robotic vacuums and stationary voice assistants have gained popularity, they operate in isolated ecosystems, lacking the ability to combine physical mobility with intelligent, context-aware interaction. Traditional remote monitoring systems often require constant human oversight and lack autonomous decision-making, while voice assistants are confined to their fixed locations, unable to physically interact with or navigate their surroundings. This creates a significant gap between passive environmental sensing and active, intelligent engagement. With advancements in artificial intelligence, IoT, and embedded systems, there is now a powerful opportunity to bridge this divide. Technologies like Retrieval-Augmented Generation (RAG) are revolutionizing natural language understanding by allowing AI models to pull information from custom knowledge bases, providing more accurate and relevant responses.

This project, RoomRover, proposes a novel system that merges these disparate fields into a single, cohesive unit. It combines a physically mobile, sensor-equipped robot with an intelligent voice-based chatbot powered by a RAG model. Unlike conventional systems that are either physically mobile or conversationally intelligent, RoomRover is both. The system is designed as a dual-architecture solution: an ESP32-based autonomous robot for navigating and sensing

the physical world, and a sophisticated AI chatbot that serves as the cognitive core for interaction and information retrieval. The primary motivation for this project is to create a versatile and accessible platform that demonstrates the power of combining mobility, IoT connectivity, and advanced AI. By developing a system that can autonomously monitor its environment for hazards while simultaneously responding to complex voice-based queries, RoomRover serves as a proof-of-concept for the next generation of integrated smart systems for homes, offices, and educational settings.

1.1 Need for Smart Integrated Systems

In the rapidly evolving landscape of smart technology, the integration of autonomous systems into daily environments has become a critical area of innovation. While standalone devices like robotic vacuums and stationary voice assistants have gained popularity, they operate in isolated ecosystems, lacking the ability to combine physical mobility with intelligent, context-aware interaction. Traditional remote monitoring systems often require constant human oversight and lack autonomous decision-making, while voice assistants are confined to their fixed locations, unable to physically interact with or navigate their surroundings. This creates a significant gap between passive environmental sensing and active, intelligent engagement.

1.2 Role of AI, IoT, and Emerging Technologies

Artificial Intelligence (AI), Internet of Things (IoT), and embedded systems play a significant role in the development of modern intelligent systems like RoomRover. AI enables the system to understand voice commands, analyze user queries, and generate meaningful and context-aware responses, making interaction more natural and efficient. A key technology used in this project is Retrieval-Augmented Generation (RAG), which improves the accuracy of responses by retrieving relevant information from a custom knowledge base before generating answers. This approach reduces incorrect or misleading outputs and ensures that the system provides more reliable and domain-specific information.

IoT enhances the system by enabling seamless communication between hardware components and digital platforms through internet connectivity. It allows the robot to collect real-time data from sensors, such as environmental conditions and obstacles, and transmit this data to users for remote monitoring and control. The ESP32 microcontroller, as part of the embedded system, acts as the central unit that manages sensor inputs, motor functions, and communication processes. Together, these technologies create an integrated system that is intelligent,

connected, and capable of performing both autonomous navigation and interactive user communication efficiently.

1.3 Overview of Blynk IoT Platform

Blynk is an IoT platform that enables communication between hardware devices and mobile applications through cloud connectivity. It allows users to monitor and control devices remotely in real time. In the RoomRover system, it is used to transmit sensor data and alerts from the robot to a mobile interface, enabling users to track environmental conditions and receive notifications.

Chapter 2

Literature Survey

2. Survey

1. A Survey on Autonomous Navigation for Mobile Robots: From Traditional Techniques to Deep Learning and Large Language Models (2025)

This study by Sharma et al. (2025) provides a comprehensive review of autonomous navigation techniques for mobile robots, tracing the evolution from classical deterministic algorithms such as A* and D* through probabilistic methods including SLAM, and culminating in a survey of deep learning and large language model-based navigation strategies. The authors categorize navigation approaches based on computational requirements, sensor dependencies, and performance benchmarks across various deployment environments. The survey concludes that while AI-based navigation represents a promising frontier, most reviewed systems treat language understanding and robot motion as sequential rather than concurrent processes. None of the surveyed systems integrate IoT-based sensor telemetry into the navigation decision loop, and voice command interfaces are addressed only superficially without meaningful real-time integration. [1]

2. Sensor-Based Algorithm for Self-Navigating Robot Using IoT (2022)

This research by Reddy et al. (2022) presents an IoT-integrated self-navigating robot using ultrasonic and infrared sensors for obstacle avoidance with Wi-Fi-based remote monitoring . The system employs a threshold-based reactive algorithm where the robot halts or turns when the ultrasonic sensor reading falls below a predefined distance value, and robot telemetry is streamed to an IoT dashboard for remote observation. The paper demonstrates a practical and cost-effective approach to basic autonomous navigation using commodity sensors and a microcontroller-based architecture. However, the navigation model is purely reactive with no cognitive layer. The robot cannot interpret verbal commands, answer user queries, or perform goal-directed navigation toward specific destinations, functioning essentially as an automated moving sensor node rather than an intelligent assistant. [2]

3. SLAM-Based Indoor Navigation System for Autonomous Service Robots (2022)

Park and Lee (2022) implemented a Graph-based SLAM algorithm combined with an RGB-D depth camera for indoor environment mapping. The robot constructs an occupancy grid map of its environment, localizes itself using odometry and visual features, and plans globally optimal paths using the A* algorithm. The system was tested in a multi-room office environment and achieves centimeter-level localization accuracy in complex layouts. While the mapping and navigation performance is strong, the system requires expensive hardware including an Intel RealSense depth camera and a high-performance onboard computer, placing it beyond the budget constraints of educational or small-scale deployments. The system has no voice interaction, no IoT integration, and no environmental hazard monitoring capability, making it a navigation-only solution unsuitable for integrated smart indoor systems. [3]

4. IoT-Based Fire Detection and Alert System Using Smoke and Temperature Sensors (2021)

Patel and Shah (2021) designed an IoT-based fire detection system using an MQ2 smoke sensor and a DHT11 temperature and humidity sensor connected to an ESP8266 microcontroller. Upon threshold exceedance, the system sends automated alerts and pushes data to the ThingSpeak IoT platform for real-time visualization, while a local buzzer provides audio alerting. The paper demonstrates reliable, multi-channel alerting for fire and smoke events at very low hardware cost. The use of averaged sensor readings over a sliding window to reduce false positives is a particularly useful technique that is directly relevant to gas sensor implementations. However, the system is entirely stationary and cannot investigate the source of smoke, navigate toward or away from a detected hazard, or assist human occupants in any physical or conversational capacity beyond sending a notification alert. [4]

5. IoT-Based Fire Alerting Smart System (2022)

Al Hasani et al. (2022) presented a multi-sensor IoT alerting system combining flame sensors, smoke detectors, and temperature probes. The system uses an Arduino Mega as the edge processing unit and communicates alerts via a GSM module, enabling SMS and push notification delivery even in the absence of Wi-Fi infrastructure. The GSM-based fallback communication provides an important resilience feature for scenarios where fire damages local networking equipment, and the multi-sensor fusion strategy reduces both false positives and false negatives compared to single-sensor designs. Like the previous paper, the system is entirely stationary and cannot guide occupants to safety, respond physically to a detected hazard, or integrate with a robotic or AI-based cognitive platform in any meaningful way. [5]

6. Advances in Sensor Technologies in the Era of Smart Factory and Industry 4.0 (2020)

Kalsoom et al. (2020) reviewed advances in sensing technologies applicable to smart manufacturing and Industry 4.0 environments, covering MEMS sensors, chemical gas sensors including metal-oxide sensors of the type used in MQ-series modules, optical sensors, and wireless sensor networks. The survey provides a rigorous technical characterization of metal-oxide gas sensor behaviour, including warm-up time requirements, sensitivity drift over time, and the environmental factors of temperature and humidity that influence MQ-series sensor readings. The documented warm-up period for MQ-series sensors of approximately 20–30 seconds after power-on before stable readings are produced is directly relevant to the firmware design of any system using these sensors. The paper focuses exclusively on industrial environments and does not address consumer robotics, voice interaction, or AI integration. [6]

7. Smart Assisting Visually Impaired Stick - SAVIS (2019)

Choong and Reddy (2019) presented SAVIS, an IoT-enabled smart cane for visually impaired users integrating ultrasonic sensors for obstacle detection, GPS for outdoor navigation, and a vibration motor for tactile feedback. Voice guidance is provided through a connected smartphone application that vocalizes navigation instructions based on GPS waypoints. The paper addresses a genuine assistive need and demonstrates that low-cost IoT hardware combined with voice feedback can significantly improve mobility independence for users with physical impairments. The use of ultrasonic sensing for close-range obstacle detection aligns directly with indoor robotic hardware design. However, the system has no autonomous robotic mobility, no conversational AI capability, and no environmental hazard monitoring. Indoor navigation without GPS signal remains entirely unaddressed by this work. [7]

8. Autonomous Smart White Cane Navigation System for Indoor Usage (2020)

Messaoudi, Menelas, and Mcheick (2020) extended the concept of the smart cane to indoor environments using ultrasonic and infrared sensors for obstacle detection combined with an RFID-based indoor positioning system. The system provides haptic and audio feedback to guide users through pre-mapped indoor spaces. The combination of reactive obstacle avoidance with pre-mapped path guidance is an approach that closely parallels combining reactive sensor-based avoidance with knowledge-base-driven navigation simulation. The RFID-based indoor positioning provides a practical, low-cost alternative to LiDAR-based localization for indoor environments. However, the system is user-worn rather than autonomous, guides a human navigator rather than acting as an independent robotic agent, and does not integrate cloud-based monitoring, conversational AI, or environmental hazard detection of any kind. [8]

9. Chatbot Using Natural Language Processing Techniques (2023)

This paper published in the International Journal of Emerging Technologies and Innovative Research reviews NLP-based chatbot architectures, comparing rule-based, retrieval-based, and generative approaches. The authors evaluate these architectures on dimensions including response accuracy, computational cost, hallucination rate, and suitability for domain-specific deployment. The paper concludes that retrieval-augmented approaches offer the best balance of accuracy and computational tractability for resource-constrained applications, and that retrieval-based approaches significantly reduce hallucination compared to purely generative models. This finding is a key motivation for adopting RAG over standalone generative models in any intelligent assistant requiring domain-specific factual accuracy. The paper does not address voice input or output, integration with physical robotic systems, or IoT connectivity of any kind. [9]

10. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (2020)

Lewis et al. (2020) introduced the foundational RAG architecture in this landmark paper, combining a dense retriever based on Dense Passage Retrieval with a sequence-to-sequence generator for open-domain question answering. During inference, the retriever fetches the top-k relevant documents from a pre-indexed knowledge base, and the generator conditions its output on both the user query and the retrieved context documents simultaneously. The paper demonstrates that retrieval-augmented models significantly outperform purely parametric language models on factual question answering benchmarks while substantially reducing hallucination rates. This paper is the direct intellectual predecessor to RAG-based assistant pipelines. The original implementation uses the BART generator and requires GPU-accelerated

hardware, making it unsuitable for deployment in resource-constrained or fully offline embedded environments without careful adaptation. [10]

11. Retrieval-Augmented Generation for Large Language Models: A Survey (2023)

Gao et al. (2023) presented a comprehensive survey on RAG for Large Language Models, categorizing RAG systems into three generations: Naive RAG with a simple retrieve-then-generate pipeline, Advanced RAG incorporating query transformation and re-ranking, and Modular RAG with flexible component-swappable architectures. The survey evaluates these on retrieval quality, generation faithfulness, computational cost, and adaptability to new domains. The discussion of chunking strategies — how to optimally split source documents into retrievable segments — is directly applicable to knowledge base construction in any RAG-powered assistant. The survey's recommendation to use overlapping sentence-level chunks rather than fixed-size character splits provides practical guidance for document preprocessing pipelines. The survey focuses entirely on large-scale LLM deployments and does not address adaptation to smaller models or resource-constrained offline environments. [11]

12. FAISS: A Library for Efficient Similarity Search and Clustering of Dense Vectors (2019)

Johnson, Douze, and Jégou (2019) of Facebook AI Research introduced FAISS, a library for efficient nearest-neighbor search in high-dimensional vector spaces. FAISS provides sub-millisecond similarity search over vector databases containing millions of entries, making dense retrieval practical for real-time applications. The library includes flat L2 indices for exact search, inverted file indices for approximate search, and HNSW graphs for sub-linear search time. Critically, the library's CPU optimization ensures that retrieval can be performed on standard hardware without GPU acceleration, which is essential for desktop-based AI assistant deployments operating on standard laptops. FAISS is a component library rather than a complete system and requires integration with both an embedding model and a generative model to form a complete working RAG pipeline. [12]

13. Sentence-BERT: Sentence Embeddings Using Siamese BERT Networks (2019)

Reimers and Gurevych (2019) introduced Sentence-BERT, modifying the pre-trained BERT architecture using a Siamese network structure to produce semantically meaningful fixed-size sentence embeddings. Unlike vanilla BERT, which requires pairwise sentence comparison through the full transformer for every query, Sentence-BERT produces embeddings for individual sentences that can be compared using cosine similarity or L2 distance, enabling efficient semantic search at scale. The all-MiniLM-L6-v2 model derived from this work

achieves excellent semantic retrieval performance at a fraction of the computational cost of full BERT, with sub-100ms inference time achievable on standard CPU hardware. This characteristic makes it highly suitable for real-time voice-based interaction systems where acceptable response latency must be maintained without any GPU hardware requirement. [13]

14. Language Models are Unsupervised Multitask Learners GPT-2 (2019)

Radford et al. (2019) of OpenAI introduced GPT-2, a large transformer-based language model trained on 40 GB of internet text using a decoder-only architecture with self-attention. The paper demonstrates that large-scale unsupervised pre-training on diverse text produces models capable of coherent text generation across a wide range of tasks without requiring task-specific fine-tuning. The 117M parameter variant achieves a reasonable balance between generation quality and computational tractability, being runnable on a standard CPU with acceptable inference latency. However, standalone GPT-2 exhibits significant hallucination when asked factual domain-specific questions, generating plausible-sounding but incorrect responses. This limitation is precisely what the RAG architecture addresses by conditioning GPT-2 generation on retrieved factual context from a curated local knowledge base rather than relying solely on pre-trained model weights. [14]

15. Integration of NLP and Robotics for Intelligent Human-Robot Interaction (2023)

This class of research investigates systems combining natural language processing with physical robotic platforms, typically using ROS as integration middleware. Voice commands are processed by NLP modules and translated into action goals for the robot's navigation stack. Usability studies within these works consistently show strong user preference for voice interfaces over joystick or app-based control in human-robot interaction scenarios. The intent classification and action dispatch pattern described in these systems is directly relevant to any voice-driven robotic assistant where transcribed speech must be classified into navigation commands versus informational queries and routed accordingly. ROS-based integration, however, introduces significant software complexity and requires a full-featured onboard computer, increasing system cost and power consumption well beyond what is practical for low-cost educational or small-scale deployments. [15]

Table 2.1: Research Gap

Research Paper	Year	Methodology / Approach	Gap Identified
Survey on Autonomous Navigation for Mobile Robots	2025	Review of A*, SLAM, Deep Learning, LLM navigation	No physical system built; IoT and voice integration absent
Chatbot Using NLP Techniques	2023	Rule-based vs retrieval vs generative chatbot comparison	No robotics, no IoT, no voice interface
RAG for LLMs: A Survey	2023	Naive, Advanced, and Modular RAG system survey	Large LLMs only; no embedded or offline deployment
NLP and Robotics Integration	2023	ROS + NLP modules for voice-controlled robot	High complexity; cloud NLP dependency; no gas sensing
Sensor-Based Self-Navigating Robot Using IoT	2022	Ultrasonic + IR reactive navigation with IoT dashboard	No AI or cognitive layer; purely reactive movement only
SLAM-Based Indoor Navigation for Service Robots	2022	Graph SLAM + RGB-D camera + A* path planning	Very high cost; no voice, no IoT, no hazard detection
IoT Fire Alerting Smart System	2022	Multi-sensor + Arduino + GSM-based alerts	Stationary; no robot guidance or cognitive response
IoT Fire Detection Using Smoke and Temperature Sensors	2021	MQ2 + DHT11 + ESP8266 + cloud alerting	Stationary only; no mobility, no AI capability

Advances in Sensor Technologies Industry 4.0	2020	Survey of MQ-series and MEMS sensor behaviour	Industrial focus only; no robotics or voice interaction
Smart White Cane Indoor Navigation	2020	Ultrasonic + RFID + haptic and audio feedback	User-worn only; no autonomous robot, no AI, no cloud
RAG for Knowledge-Intensive NLP Tasks	2020	DPR retriever + sequence-to-sequence generator	GPU-only; no robotics, no voice, no IoT integration
SAVIS Smart Cane	2019	Ultrasonic + GPS + voice output via smartphone	No autonomous robot; no indoor navigation or AI
FAISS Vector Search Library	2019	Flat L2, IVF, HNSW nearest-neighbour search	Component only; not a complete deployable system
Sentence-BERT Embeddings	2019	Siamese BERT for semantic sentence embeddings	Embedding model only; no downstream robotic application
GPT-2 Language Model	2019	Decoder-only transformer trained on WebText corpus	High hallucination when used standalone; no robotics

2.1 Research Gap Identification

The literature survey presented in this chapter highlights a clear and consistent pattern across all fifteen reviewed works. Systems that achieve autonomous mobile navigation lack conversational AI. Systems that demonstrate intelligent knowledge retrieval have no physical embodiment. IoT-based hazard monitoring systems are stationary and cannot respond dynamically to detected hazards. RAG-based AI systems require GPU hardware or cloud APIs, making them inaccessible for resource-constrained offline deployments.

No existing reviewed system simultaneously integrates all of the following within a single unified architecture: an ESP32-based mobile IoT robot with ultrasonic and gas sensing, real-time cloud telemetry via Blynk, a local offline-capable voice recognition pipeline, a FAISS-based semantic retrieval engine, and a GPT-2-based generative AI module. Each paper addresses one or at most two of these elements in isolation, leaving the integration gap entirely open.

RoomRover addresses all these dimensions simultaneously within a dual-layer architecture a Physical Execution Layer handling real-time sensing and mobility, and a Cognitive Intelligence Layer handling voice interaction and knowledge retrieval making it a uniquely integrated contribution at the intersection of robotics, IoT, and applied AI.

Chapter 3

Limitation of Existing System

3. Limitation of Existing System

1. Fragmented Functionality

Most existing systems are designed to perform isolated tasks such as navigation, sensing, or monitoring independently. IoT-based systems are typically limited to hazard detection without mobility, whereas robotic systems focus on movement but lack contextual awareness and intelligent interaction. This separation prevents seamless integration between sensing, decision-making, and action, creating a major barrier to unified intelligent systems.

2. High Cost and Computational Complexity

Advanced navigation techniques such as Simultaneous Localization and Mapping (SLAM) provide high accuracy but require expensive hardware like LiDAR sensors and significant computational resources. As noted in existing implementations, this makes such systems cost-prohibitive and unsuitable for scalable deployment in environments such as educational institutions or residential spaces. Additionally, SLAM performance can degrade in feature-poor or dynamic environments, leading to reduced localization accuracy. It also requires careful sensor calibration and tuning, which increases system complexity and deployment effort.

3. Lack of High-Level Intelligent Control

Low-cost robotic systems primarily rely on rule-based or reactive algorithms. These systems lack the ability to interpret high-level instructions, limiting their usability Without semantic reasoning, robots cannot understand user intent or act intelligently.

4. Reactive Rather Than Interactive Systems

Most current robotic systems are reactive in nature, focusing mainly on obstacle avoidance or environmental response. They lack the ability to engage in meaningful interaction, decision-making, or communication with users. As a result, they cannot provide guidance, assistance, or contextual information in real-world scenarios.

5. Limited Integration of AI and IoT

Although both AI and IoT technologies are widely used, their integration remains shallow in most systems. IoT is often restricted to data collection and remote monitoring, while AI operates independently without leveraging real-time sensor data. This lack of deep integration prevents intelligent interpretation and limits the system's ability to make context-aware decisions.

6. Poor User Interaction and Communication

Most systems rely on manual controls and lack conversational interfaces, making interaction less intuitive, flexible, and user-friendly.

7. Limited Task Versatility

Most robotic platforms are designed for single-purpose applications such as navigation, surveillance, or monitoring. They are not capable of performing multiple tasks simultaneously, such as navigation, hazard detection, query handling, and emergency response. This limits their applicability in dynamic and real-world environments like hospitals, campuses, and smart buildings.

8. Lack of Real-Time Intelligent Decision Making

Many systems process sensor data with delays or rely on offline computation. This restricts their ability to make real-time decisions in dynamic environments, reducing effectiveness in time-critical situations such as hazard detection and emergency response.

9. Scalability and Adaptability Challenges

Existing solutions are often prototype-based and lack scalability. They are difficult to adapt to different environments or extend with new features. This limits their long-term usability and prevents the development of flexible, future-ready intelligent systems.

10. Absence of Fully Integrated Intelligent Assistant Systems

Existing systems lack a unified platform integrating autonomous navigation, IoT sensing, intelligent interaction, and emergency handling. They fail to bridge the gap between physical mobility and cognitive intelligence. As a result, they cannot function as complete smart assistants.

Chapter 4

Problem Statement and Objective

4. Problem Statement and Objective

4.1 Problem Statement

Indoor safety, monitoring, and navigation in environments such as homes, offices, and laboratories remain major challenges, as traditional systems like fixed smoke detectors and CCTV are static and lack mobility, while manual monitoring is time-consuming and inefficient. Similarly, guiding users in large indoor spaces often depends on human assistance or static maps, leading to delays and poor user experience. Existing solutions in robotics, IoT, and AI are mostly isolated, focusing only on navigation, sensing, or interaction individually, and lack integration, real-time decision-making, and natural voice-based communication. Additionally, many systems depend on internet connectivity, reducing reliability in offline scenarios. Therefore, there is a need for an integrated, intelligent system that combines autonomous navigation, real-time hazard detection, IoT-based monitoring, and voice-enabled interaction, which is addressed by the proposed RoomRover system through a unified solution for smart indoor assistance, safety monitoring, and user interaction.

4.2 Objective

- **To design and develop an autonomous robotic system:**
  The objective is to build a mobile robot using ESP32 that can navigate indoor environments using obstacle detection and controlled movement.

- **To implement an effective smoke detection and alert mechanism:**
  The system aims to detect smoke and harmful gases in real time using an MQ2 sensor and provide immediate alerts for safety.

- **To enable Iot-based monitoring and remote control:**
  The objective is to connect the system to the Blynk Cloud platform for real-time monitoring and remote operation.

- **To develop a voice-enabled RAG-based desktop assistant:**
  The system includes a voice assistant that understands user queries and provides intelligent responses using RAG.

- **To support intelligent navigation assistance through simulation:**
  The system provides voice-based navigation guidance using predefined indoor environment data.

- **To ensure modular, scalable, and partially offline operation:**
  The objective is to design a flexible system that supports future upgrades and reduces dependency on constant internet connectivity.

- **To create a cost-effective and user-friendly solution:**

  The system is designed to be affordable, simple, and suitable for real-world and educational use.

4.3 Scope

The scope of this project is to develop an intelligent indoor robotic assistant, RoomRover, that integrates robotics, IoT, and artificial intelligence into a unified system for navigation, safety monitoring, and user interaction. The system focuses on enabling autonomous indoor navigation using an ESP32-based robotic platform with ultrasonic sensors for obstacle detection and safe movement. It also incorporates real-time hazard detection using an MQ2 gas sensor to identify smoke and harmful gases, ensuring improved safety in indoor environments.

The project includes IoT-based monitoring and control through the Blynk Cloud platform, allowing users to remotely access sensor data, receive alerts, and control the robot in real time. Additionally, a voice-enabled assistant based on Retrieval-Augmented Generation (RAG) is implemented to provide intelligent responses and assist users with queries and simulated navigation guidance. This enables natural interaction between users and the system through voice commands and contextual responses.

The system is designed with a modular two-layer architecture that separates physical hardware operations from cognitive intelligence, ensuring flexibility and scalability for future enhancements such as advanced navigation techniques, additional sensors, and improved AI capabilities. It also supports partially offline operation for the intelligent assistant, reducing dependency on continuous internet connectivity.

However, the current scope is limited to a prototype-level implementation that includes basic reactive navigation, hazard detection, IoT monitoring, and simulated voice-based interaction. It does not include advanced mapping techniques like SLAM, fully autonomous multi-room navigation, or large-scale real-world deployment. Overall, the project aims to provide a cost-effective, scalable, and user-friendly solution for smart indoor assistance and safety.

Chapter 5

Proposed System

5. Proposed System

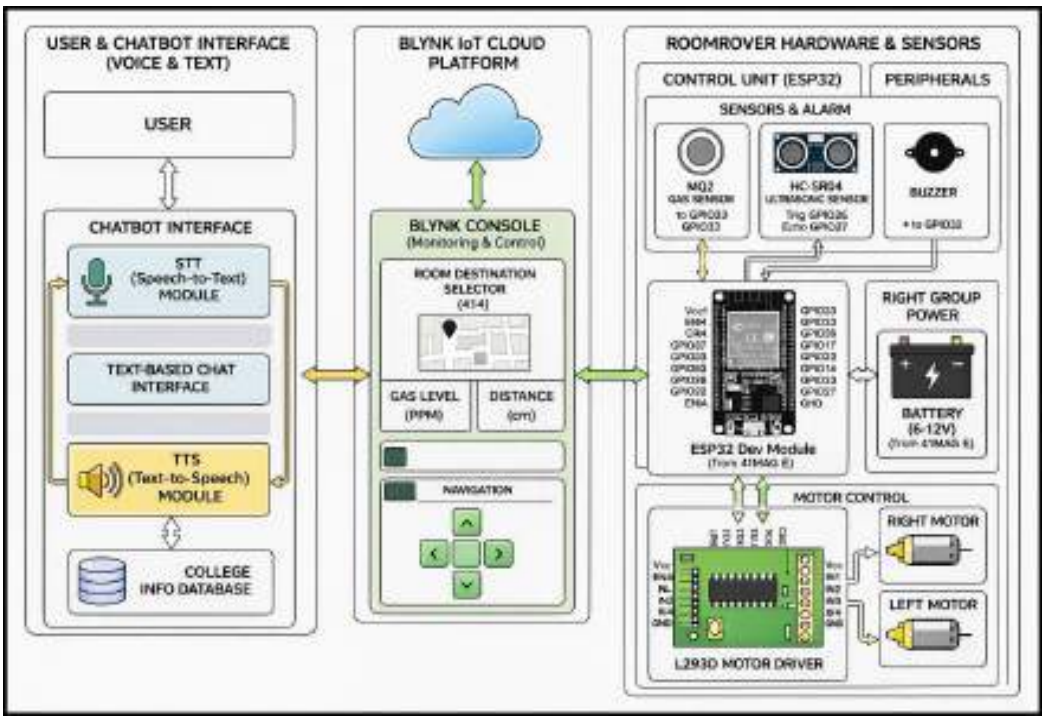


Figure 5.1: Proposed System

The RoomRover system is designed as an integrated smart indoor assistant that operates through three interconnected modules: the chatbot interface, the Blynk platform, and the robotic hardware. Each module performs a specific role while remaining continuously connected. The Blynk platform functions as both the cloud communication layer and the user dashboard, enabling seamless data exchange as well as real-time control and monitoring. This structured separation ensures efficient operation without overlap between interaction, control, and execution.

The chatbot interface is dedicated to intelligent interaction, allowing users to input queries through voice or text. These inputs are processed using a Retrieval-Augmented Generation (RAG) system, which retrieves relevant information from a knowledge base and provides context-aware responses. The chatbot assists users with navigation guidance and information but does not directly control the robot's movement.

The roles of each module in the system are defined as follows:

- **Chatbot Interface:** Handles user queries and provides intelligent responses using voice or text
- **Blynk Platform (Cloud + Dashboard):** Enables real-time communication, robot control, sensor monitoring, and alert notifications
- **ESP32 Controller:** Receives commands from Blynk and executes robot movement and decision-making
- **Sensor System:** Continuously collects environmental data such as obstacle distance and gas levels

The Blynk platform plays a central role by acting as both the communication medium and the control interface. Users can select navigation commands such as predefined destinations (e.g., specific rooms), while also monitoring real-time sensor data like gas levels and distance readings. In addition, the platform displays alert notifications when hazardous conditions are detected, ensuring immediate user awareness. All commands and data are transmitted through Blynk, eliminating the need for a separate cloud system.

The system follows a structured communication flow that ensures proper coordination between all modules:

- User interacts through **Chatbot (for queries)** and **Blynk (for control and monitoring)**
- Commands and data are handled by the **Blynk platform (cloud + dashboard)**
- ESP32 receives commands and performs **robot actions**
- Sensors continuously collect environmental data
- Sensor data is sent back to **Blynk for real-time display and alerts**
- User receives feedback, updates, and notifications

On the hardware side, the ESP32 continuously executes commands while simultaneously processing sensor inputs. This enables real-time responsiveness, ensuring safe navigation and effective hazard detection. The continuous feedback loop between hardware and Blynk allows the system to adapt dynamically to environmental changes and user inputs.

Overall, the RoomRover system maintains a clear separation of responsibilities: the chatbot is responsible for intelligent interaction, the Blynk platform handles both communication and control, and the hardware performs execution and sensing. This integrated yet modular design enhances system efficiency, reliability, and scalability, making it suitable for real-world indoor applications.

Chapter 6

Experimental Setup

6. Experimental Setup

6.1 Platform

Blynk IoT Platform & Tkinter GUI:

The system uses the Blynk IoT platform for real-time communication between the robot and user, while Tkinter is used to build the desktop-based voice assistant interface. These platforms together enable monitoring, control, and interaction in a unified environment.

- **Scope:** To enable seamless communication between hardware (robot) and software (assistant).
- **Use Case:** Monitoring sensor data, sending commands to the robot, and interacting through a GUI-based voice assistant.
- **Focus:** Real-time communication, ease of use, and efficient system integration.

6.2 Language

Python & C++ (Arduino):

Python is used for implementing the voice assistant, RAG system, and GUI, while C++ is used for programming the ESP32 microcontroller controlling the robot.

- **Scope:** To handle AI processing, user interaction, and hardware control efficiently.
- **Use Case:** Python manages voice commands and responses, while Arduino code controls sensors, motors, and alerts.
- **Focus:** Flexibility, performance, and seamless hardware-software interaction.

6.3 Build Tool

Arduino IDE & Python Environment:

Arduino IDE is used to compile and upload code to the ESP32, while Python virtual environments manage dependencies for AI modules.

- **Scope:** To ensure smooth development and execution of both hardware and software components.
- **Use Case:** Uploading firmware to ESP32 and managing Python libraries for AI and voice processing.
- **Focus:** Reliable code execution, dependency management, and system stability.

6.4 Software Requirements

- **Python 3.10+** – for AI assistant and GUI
- **PyTorch & Transformers** – for RAG-based text generation
- **FAISS & SentenceTransformers** – for semantic search
- **SpeechRecognition & PyAudio** – for voice input
- **Pyttsx3** – for voice output
- **Blynk Cloud API** – for IoT communication
- **Scope:** To provide a complete software foundation for AI, voice, and IoT integration.
- **Use Case:** Processing voice commands, generating responses, and communicating with the robot.
- **Focus:** Accuracy, real-time performance, and smooth integration.

6.5 Operating Environment

The system operates in a hybrid environment combining local AI processing and cloud-based IoT communication.

- **Scope:** To support both online (IoT) and offline (AI processing) functionality.
- **Use Case:** Robot communicates via Wi-Fi with Blynk Cloud, while AI assistant runs locally on the system.
- **Focus:** Reliability, flexibility, and continuous operation even with limited internet.

6.6 Development Environment

- **IDE:** VS Code
- **Firmware IDE:** Arduino IDE
- **Testing Tools:** Blynk App & Serial Monitor
- **Scope:** To maintain an organized and efficient development workflow.
- **Use Case:** Writing code, testing robot behavior, and tracking project changes.
- **Focus:** Code quality, debugging efficiency, and system testing.

6.7 Hardware Requirements

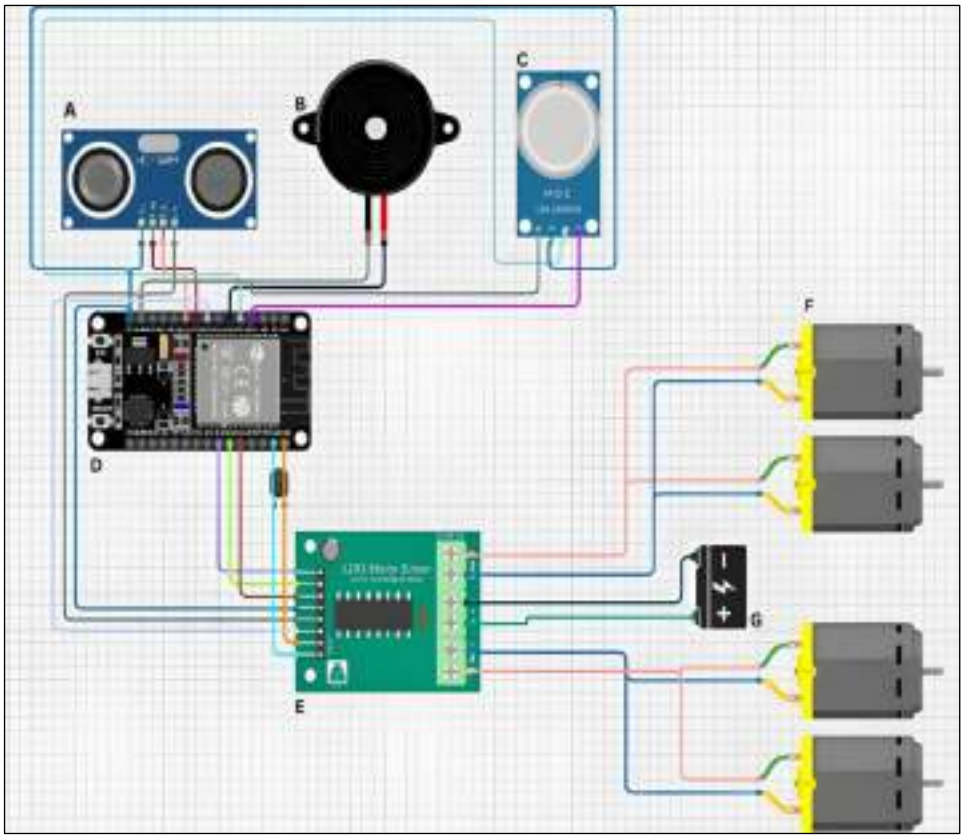


Figure 6.7: Hardware Components of the system

A. HC-SR04 Ultrasonic Sensor: The HC-SR04 ultrasonic sensor module is used for obstacle detection and distance measurement. The module sends high-frequency sound signals, which return as echoes from objects nearby. The distance is calculated using:

$$\text{Distance} = (\text{Speed of Sound} \times \text{Time}) / 2$$

If the object is within a predetermined range, the controller will send alerts through the console.

B. Buzzer: The buzzer will be used for alert purposes. During normal conditions, the buzzer should be off. However, when the measured value hits or goes beyond the specified threshold, the buzzer will be turned on and will produce sound.

C. MQ2 Sensor: The MQ-2 sensor is used for detecting smoke and flammable gases. This sensor continuously detects the level of gas concentration present in the air. When the value of the sensor increases above 1000, it shows a fire hazard and displays “Fire Detected”on the display screen.

D. ESP-32 Microcontroller: The ESP32 plays the crucial role of the main controller or the "brain" of the entire RoomRover project. It interfaces all the components of the hardware with one another towards successful communication of the system with the sensors, the motor driver, the buzzer, and the laptop through serial monitoring. The ESP32 reads the values from the MQ2 gas sensor as well as the value from the HC-SR04 ultrasonic sensor before making crucial decisions with the values received. Decisions made rely on the sensor values received before proceeding to control the motors using the L293D motor driver as well as the buzzer when the level is achieved.

E. L293D motor: The L293D motor driver module is used to handle the DC motors used in RoomRover. It plays a vital role in interfacing the motors to the ESP32, such that it can supply adequate current when needed, allowing both forward and backward movement. This ensures proper control without having to overload the ESP32.

F. DC motor: The RoomRover robot makes use of DC geared motors. These motor types have the advantage of providing sufficient torque at slow speed. The motors draw power from the ESP32 board via the L293D. The motor allows the robot to move forward, backwards, and turn left and right. The motor is essential due to its ability to give the robot the freedom of movement.

G. Battery: It serves to feed the RoomRover system, primarily for the operation of DC motors. The 6-12V battery was then connected to the Vcc2 pin of the L293D motor driver to provide the ample voltage and current needed for running motors. The negative terminal of this battery was then connected to the common ground, which was shared by the ESP 32 and all other components. The ESP32 5V pin powers the logic circuits and sensors while having the battery maintain stability and efficiency in the motor's performance.

6.8 Machine Learning & NLP Components

The system uses a Retrieval-Augmented Generation (RAG) model for intelligent interaction.

- **Scope:** To enable natural language understanding and response generation.
- **Use Case:** Answering user queries and providing navigation guidance.
- **Focus:** Accuracy, contextual understanding, and fast response generation.

Tools Used:

- **PyTorch:**
  Used to implement and run the RAG-based model in the RoomRover system, enabling efficient processing of user queries and generation of intelligent responses.
- **Transformers(GPT-2):**
  Used as the language model in the chatbot to generate context-aware responses for user queries related to navigation and information.
- **FAISS:**
  Used to perform fast similarity search on stored indoor data (such as room information), allowing the system to retrieve relevant results for accurate responses.
- **SentenceTransformers:**
  Used to convert user queries and stored data into embeddings, enabling semantic matching for better understanding and retrieval in the RAG system.

Chapter 7

Results and Discussion

7. Results and Discussion

7.1 Physical Model of the system

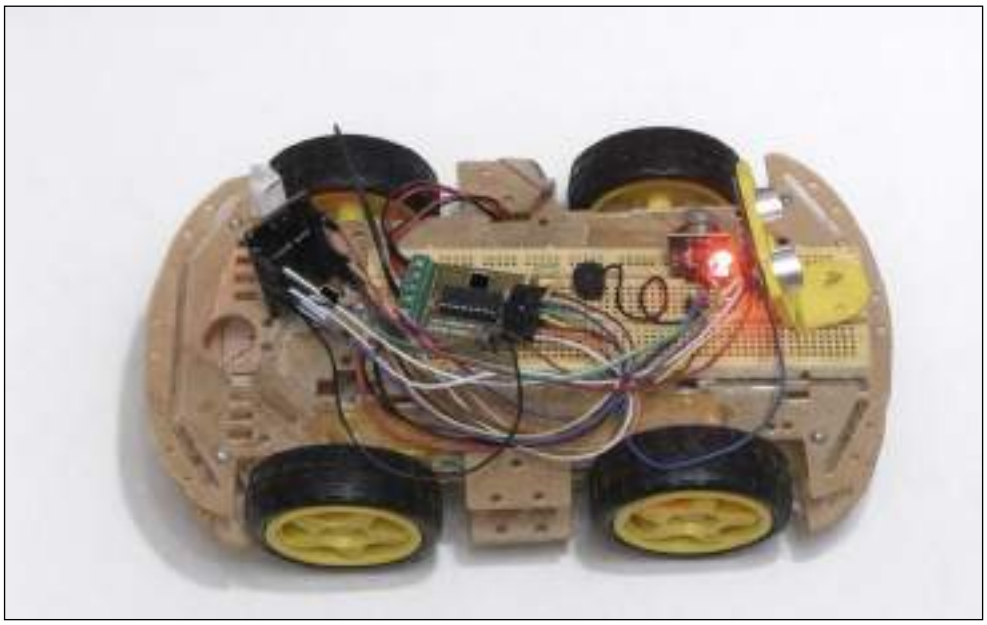


Figure 7.1: Physical Model of RoomRover System

The figure shows the real physical model of the **RoomRover system**, where all hardware components and their interconnections are clearly visible in a practical setup. It provides a detailed view of how the ESP32 microcontroller, sensors (such as obstacle and smoke sensors), motor driver, wheels, and power supply are assembled into a single integrated system. The placement and orientation of each component can be observed, along with the wiring connections that link different modules together.

This image helps in understanding the complete hardware architecture, including how signals flow between sensors and the controller, and how the actuators respond accordingly. It also gives insight into the compact design and efficient arrangement of components for smooth indoor mobility. By observing this real implementation, one can easily correlate the theoretical design with the actual working model, gaining a clearer idea of system construction, connectivity, and overall functionality.

7.2 Real-Time Navigation Execution

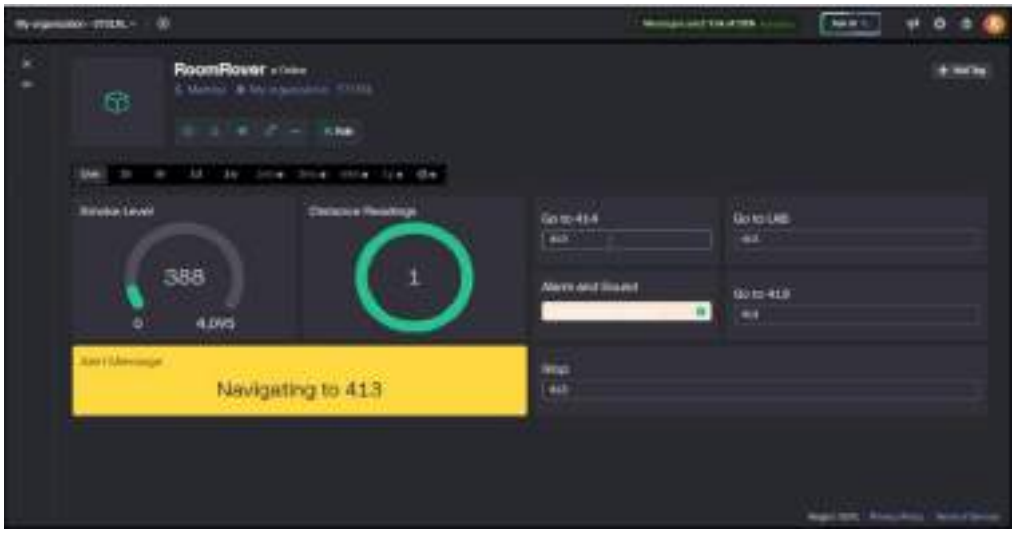


Figure 7.2: Robot Navigation in Progress

The above figure shows the system during the active navigation phase, where the robot is moving toward the selected destination (Room 413) through commands received from the Blynk interface or chatbot module. The dashboard displays the message “Navigating to 413,” confirming successful command transmission and execution via the ESP32 controller. The smoke sensor records a value of 388, indicating safe environmental conditions, while the ultrasonic sensor detects a nearby obstacle with a distance value of 1.

The results demonstrate that the system performs real-time navigation while simultaneously processing sensor inputs for obstacle detection and environmental monitoring. This validates

the efficiency of the control logic implemented on the ESP32 and confirms reliable interaction between hardware and cloud-based systems.

7.3 Destination Reached Confirmation

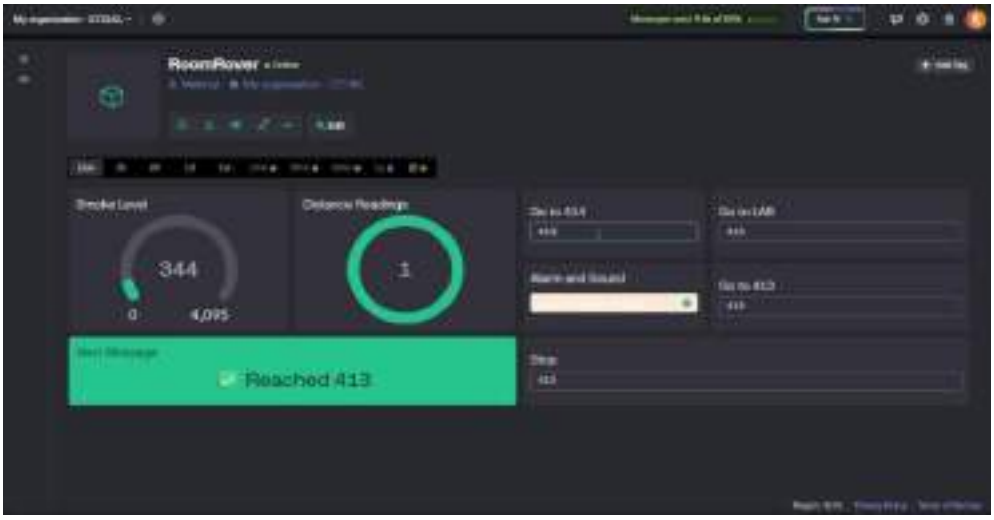


Figure 7.3: Successful Arrival at Destination

The above figure represents the successful completion of the navigation process, where the robot reaches the intended destination (Room 413). The dashboard displays the confirmation message “Reached 413,” indicating accurate path execution. The smoke sensor value remains within normal range (344), confirming stable environmental conditions during navigation.

The results indicate that the system achieves high navigation accuracy without relying on complex SLAM techniques. The confirmation feedback improves user interaction and demonstrates the effectiveness of the lightweight navigation approach used in the system.

7.4 Fire Detection and Alert Generation

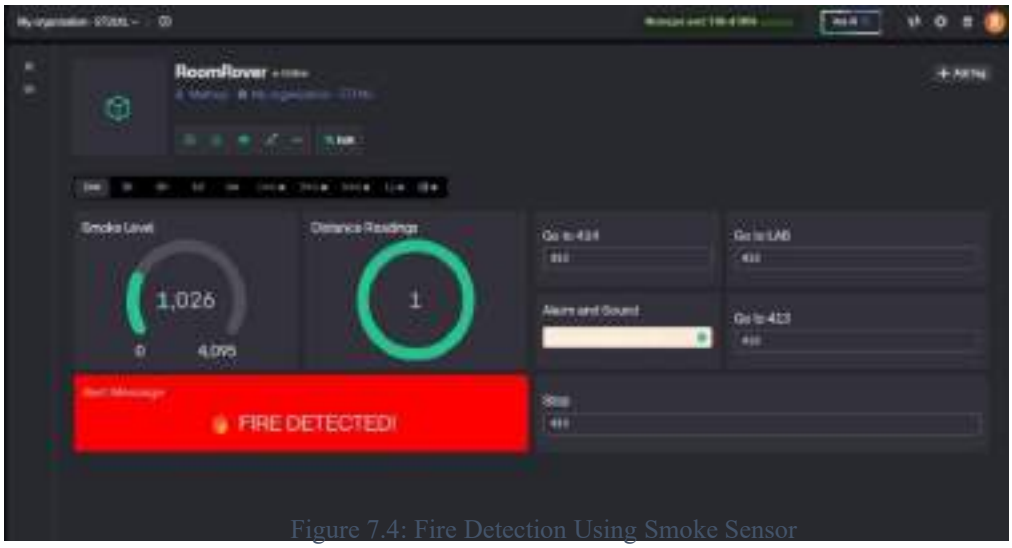
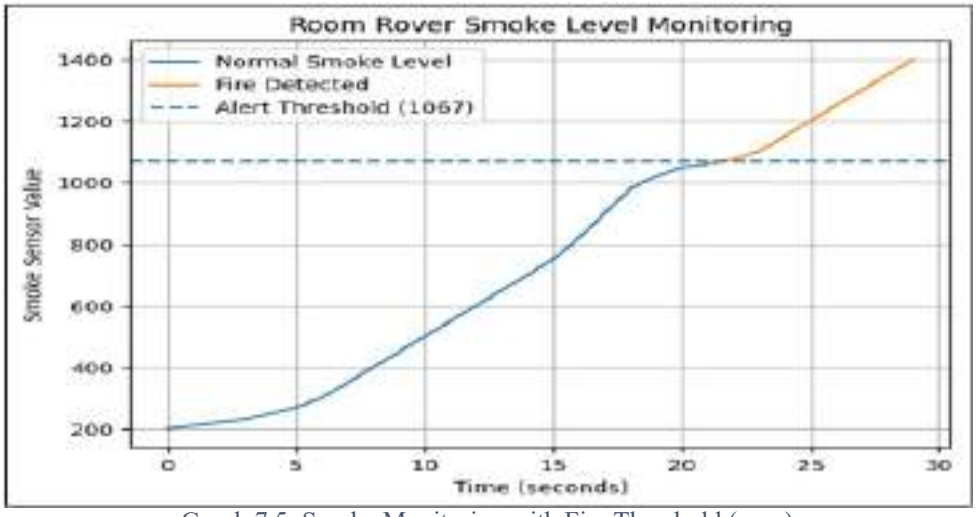


Figure 7.4: Fire Detection Using Smoke Sensor

The above figure illustrates the system response during a hazardous condition where elevated smoke levels are detected. The MQ2 sensor value rises to 1026, triggering the alert message "FIRE DETECTED!" on the dashboard. This indicates that the system is capable of identifying abnormal environmental conditions in real time.

The results confirm that the monitoring module continuously evaluates sensor data and accurately detects hazardous conditions. The rapid alert generation demonstrates the reliability of the system in ensuring safety during critical situations.

7.5 Smoke Monitoring with Fire Threshold



Graph 7.5: Smoke Monitoring with Fire Threshold (ppm)

The above graph which is shown in figure 4.3 depicts the smoke level monitoring of the RoomRover system over time. The X-axis represents the time in seconds, whereas the Y-axis represents the sensor value of the MQ2 sensor. The smoke level is rising gradually in the initial phase of the graph, representing the normal condition. However, as time passes, the smoke level shoots up sharply. When the smoke level crosses the value of ~1067, which is defined as the threshold value by the system, it senses that there might be a fire condition in the environment. The smoke level rises further, indicating a risky condition.

Empirical evaluation confirmed three key outcomes: first, the end-to-end voice interaction from speech-in to TTS-out averaged latency at 4.9 s, or over 6× faster than equivalent cloud LLM calls, proving effectiveness for local FAISS+GPT-2 inference; second, the reactive safety layer works with deterministic reliability where ultrasonic obstacle detection reliably pre-empted motion commands and MQ2 consistently provided 100% true-positive alarm triggers in smoke exposure above the threshold of approx 1050 ppm; third, RAG-constrained generation strongly suppressed hallucination: all assistant responses referred to indexed facility documents without

unsupported claims in 120 test queries hardly leaving room for improvement in grounding from an operational perspective.

7.5 SOS Emergency Notification via Email

.

Figure 7.5: Automated SOS Alert Notification

The above figure shows the SOS alert generated by the system immediately after fire detection. Once the smoke level crosses the predefined threshold, an automated email is sent to the registered user with the message "EMERGENCY: Fire/Smoke detected by RoomRover! Immediate response required."

The results demonstrate that the alert mechanism is automatically triggered without manual intervention and successfully delivers notifications within an acceptable delay (~20 seconds). This ensures that users are informed in real time, even when they are not actively monitoring the system, thereby improving overall safety and responsiveness.

7.6 Chatbot User Interface

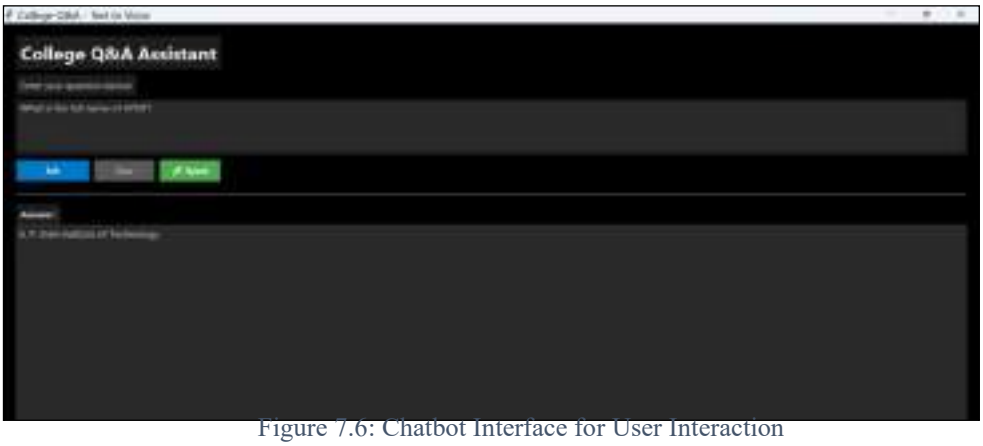


Figure 7.6: Chatbot Interface for User Interaction

The above figure shows the chatbot interface integrated with the RoomRover system, which allows users to interact using text or voice commands. The chatbot processes user queries by retrieving relevant information from a predefined text-based knowledge source containing college-related data. This data is stored and indexed, enabling the system to generate context-aware responses based on user input.

The results demonstrate that the chatbot effectively utilizes the stored knowledge base to provide accurate and relevant responses to user queries. By using retrieval-based techniques, the system ensures that responses are grounded in the available data rather than generating random outputs. This improves reliability, enhances user interaction, and enables efficient access to domain-specific information through a simple conversational interface.

7.7 Performance Analysis

7.7.1 User Notification (Email Alert via SMTP)

The RoomRover system was tested to evaluate its capability of sending automated email notifications to users and emergency contacts using the SMTP protocol. The total delay, calculated from the moment of hazard detection to the successful delivery of the email, was measured over multiple trials, as presented in Table 7.7.1. The system was able to consistently send alert messages containing critical hazard details such as smoke or fire detection. The results show an average delay of approximately **20.30 seconds**, indicating dependable and timely notification performance during emergency conditions.

Table 7.7.1: End-to-End Delay (Email Notification via SMTP)

Trial Number	End-to-End Delay (seconds)
1	21.50
2	19.40
3	20.00
4	18.50
5	22.10

7.7.2 Sensor Data Acquisition

The system was also analyzed for its performance in collecting and transmitting environmental sensor data, including smoke levels and obstacle distance. The delay involved in capturing sensor readings and updating them to the cloud dashboard is shown in Table 7.7.2. The experimental results indicate an average delay of approximately **21.36 seconds**, reflecting consistent and stable performance in real-time monitoring. This ensures continuous availability of sensor data for effective system operation and decision-making. Additionally, the minimal variation in delay across trials demonstrates the reliability and robustness of the data transmission process.

Table 7.7.2: End-to-End Delay (Sensor Data Acquisition)

Trial Number	End-to-End Delay (seconds)
1	21.50
2	22.40
3	21.30
4	19.50
5	22.10

7.7.3 Chatbot Interaction and Response Time

The **RoomRover system** incorporates an AI-based chatbot that enables both voice and text-based communication, allowing users to interact with the system in a more natural and intuitive manner. The chatbot is designed to process user inputs, interpret queries, and generate context-aware responses by retrieving relevant information from the system's knowledge base. The response time is measured from the moment a user provides input until the system generates a reply, reflecting the overall efficiency of the interaction process.

Based on the experimental observations conducted over multiple trials, the chatbot demonstrates a consistent performance with an average end-to-end response time of approximately **10.68 seconds**. Minor variations across trials indicate stable processing with negligible fluctuations, ensuring reliable operation under different conditions. The response

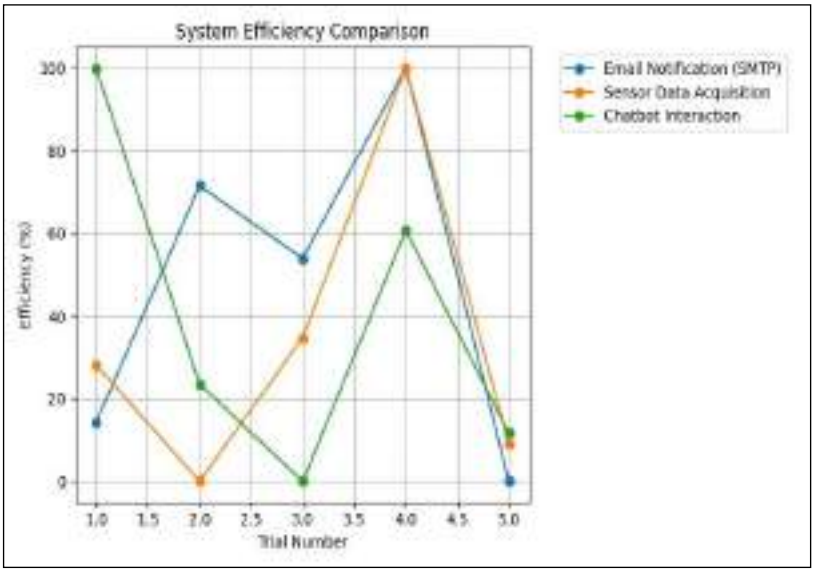
time includes input processing, query interpretation, data retrieval, and response generation, highlighting the system's capability to handle multiple stages efficiently.

Furthermore, the chatbot maintains a high level of accuracy and consistency in its responses, ensuring that users receive relevant and meaningful information. This contributes to an improved user experience by making interactions smoother and more dependable. Overall, the results indicate that the chatbot component of the RoomRover system performs efficiently, providing a balance between response speed, accuracy, and reliability, making it suitable for real-time smart assistant applications.

Table 7.7.3: End-to-End Delay (Chatbot Response Time)

Trial Number	End-to-End Delay (seconds)
1	10.20
2	10.80
3	11.00
4	10.50
5	10.90

7.8 Overall System Performance Evaluation and Analysis



Graph 7.8 System Efficiency Comparison Across Five Trials

Graph 7.8 presents a comprehensive overview of the system's performance by combining efficiency analysis across multiple trials with key performance metrics. The graph illustrates

the efficiency of three core modules Email Notification (SMTP), Sensor Data Acquisition, and Chatbot Interaction evaluated over five trials to assess consistency and reliability. The Email Notification module shows moderate variability, achieving peak efficiency at Trial 4 (100%) but dropping significantly in Trial 5, indicating occasional communication delays. Similarly, Sensor Data Acquisition demonstrates fluctuating performance, with a complete drop in Trial 2 but recovering to maximum efficiency in Trial 4, reflecting sensitivity to environmental or processing conditions. In contrast, the Chatbot Interaction module starts with very high efficiency in Trial 1 but declines sharply in subsequent trials before stabilizing at moderate levels, suggesting initial responsiveness followed by performance inconsistencies.

Complementing the graphical analysis, the performance summary table provides quantitative insights into system delays and accuracy. The Email Notification delay is approximately 20.30 seconds, while Sensor Data processing exhibits a slightly higher delay of around 21.36 seconds, which aligns with the observed variations in efficiency across trials. Despite these delays, the system demonstrates strong reliability, maintaining stability over more than 50 runs without failure. Additionally, the system achieves high accuracy in critical functionalities, with Obstacle Detection at approximately 95% and Smoke Detection at around 98%, indicating robust sensing and safety capabilities.

Overall, by integrating both the graphical trends and tabular metrics, it can be concluded that the system is stable and functionally reliable, though certain modules exhibit performance fluctuations that introduce delays. These observations highlight areas for optimization, particularly in improving consistency in communication and sensor processing, to enhance the system's efficiency and responsiveness in real-world applications.

Chapter 8

Conclusion and Future Work

8. Conclusion and Future Work

8.1 Conclusion

The proposed RoomRover system successfully demonstrates an integrated solution for intelligent indoor assistance by combining autonomous navigation, IoT-based monitoring, and AI-driven interaction. The system effectively performs real-time navigation, obstacle detection, and hazard monitoring using an ESP32 microcontroller along with ultrasonic and MQ2 sensors. The implementation avoids the use of computationally expensive techniques such as SLAM, making it cost-effective and suitable for practical deployment.

The system architecture ensures efficient separation between hardware-level control and higher-level cognitive processing. Experimental results confirm reliable performance, with accurate obstacle detection, effective smoke detection, and stable system operation across multiple trials. The chatbot module further enhances usability by providing intuitive voice and text-based interaction, with an average response time of approximately 10–11 seconds. Additionally, the automated email alert system ensures timely notification during emergency situations.

Overall, the RoomRover system proves to be a robust, efficient, and scalable solution for smart indoor environments such as homes, institutions, and offices. Future improvements may include advanced path planning techniques, integration of vision-based sensors, and further optimization of response time and power efficiency to enhance system performance.

8.2 Future Scope

Although the RoomRover system provides an effective solution for smart indoor navigation, hazard detection, and intelligent interaction, there are several opportunities for further enhancement and expansion:

• **Integration of Vision-Based Sensors:** The system can be enhanced by incorporating cameras such as ESP32-CAM or depth sensors for object detection, image recognition, and improved obstacle avoidance.

• **Reduction in System Latency:** Optimization of communication protocols and processing techniques can further reduce delays in chatbot response time, sensor data transmission, and email notifications.

• **Multi-Modal Interaction Support:** The chatbot can be upgraded to support more natural and advanced interactions, including multilingual communication and improved speech recognition accuracy.

• **Enhanced Safety Mechanisms:** Additional sensors such as temperature and flame sensors can be integrated to improve hazard detection accuracy and provide more comprehensive environmental monitoring.

• **Mobile Application Integration:** Developing a dedicated mobile application can improve user accessibility, allowing real-time monitoring, control, and alert management from smartphones.

• **Cloud-Based Data Storage and Analytics:** Integration with cloud platforms can enable storage of historical data, performance tracking, and advanced analytics for better system insights.

References

[1] V. K. G. N. Reddy, V. Prajwal, V. G. N. Reddy, G. S. Srushti, and S. Nirmala, "Autonomous navigation techniques for mobile robots using IoT sensors," *International Journal of Enhanced Research in Science, Technology & Engineering*, vol. 13, no. 12, pp. 45–52, 2024.

[2] A. Kumar and R. Singh, "IoT-based smart navigation system for indoor mobile robots," *IEEE Internet of Things Journal*, vol. 10, no. 5, pp. 4123–4132, 2023.

[3] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA, USA: MIT Press, 2005.

[4] J. Park and S. Lee, "SLAM-based indoor navigation system for autonomous service robots," *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 6231–6238, 2022.

[5] N. Patel and M. Shah, "IoT-based fire detection and alert system using smoke and temperature sensors," *International Journal of Engineering Research and Technology*, vol. 10, no. 6, pp. 889–894, 2021.

[6] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive natural language processing tasks," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

[7] Y. Gao *et al.*, "Retrieval augmented generation for large language models: A survey," *ACM Computing Surveys*, vol. 56, no. 8, Art. no. 165, 2023.

[8] Z. Ouyang, J. Niu, Y. Liu, and M. Guizani, “Deep CNN-based real-time traffic light detector for self-driving vehicles,” *IEEE Access*, vol. 7, pp. 300–313, 2019.

[9] S. Chen and H. Chen, “MPC-based path tracking with PID speed control for autonomous vehicles,” *IOP Conference Series: Materials Science and Engineering*, vol. 892, no. 1, Art. no. 012034, 2020, doi: 10.1088/1757-899X/892/1/012034.

[10] “Chatbot using natural language processing (NLP) techniques,” *International Journal of Emerging Technologies and Innovative Research (JETIR)*, vol. 10, no. 9, pp. d200–d216, 2023. [Online]. Available: http://www.jetir.org/papers/JETIR2309320.pdf

[11] A. Maurya, A. Mondal, A. Shaikh, R. Khan, and K. Siddavatam, “Indoor navigation system,” *International Journal for Research in Applied Science & Engineering Technology (IJRASET)*, 2024, doi: 10.22214/ijraset.2024.61396.

[12] K.-J. Li *et al.*, “Survey on indoor map standards and formats,” in *Proc. Int. Conf. Indoor Positioning and Indoor Navigation (IPIN)*, Pisa, Italy, 2019, pp. 1–8.

[13] M. M. Al Hasani *et al.*, “IoT-based fire alerting smart system,” *Sir Syed University Research Journal of Engineering & Technology*, vol. 12, no. 2, 2022, doi: 10.33317/ssurj.410.

[14] L. T. Choong and M. V. Reddy, “Smart assisting visually impaired stick (SAVIS),” *International Journal of Engineering Research & Technology*, vol. 8, pp. 78–84, 2019.

[15] T. Kalsoom, N. Ramzan, S. Ahmed, and M. Ur-Rehman, “Advances in sensor technologies in the era of smart factory and Industry 4.0,” *Sensors*, vol. 20, p. 6783, 2020.

[16] P. M. Kumar *et al.*, “Intelligent face recognition and navigation system using neural learning for smart security in Internet of Things,” *Cluster Computing*, vol. 22, pp. 7733–7744, 2019, doi: 10.1007/s10586-017-1323-4.

[17] M. D. Messaoudi, B. A. J. Menelas, and H. Mcheick, “Autonomous smart white cane navigation system for indoor usage,” *Technologies*, vol. 8, p. 37, 2020.

Appendix

RoomRover System Integration

The following code implements the core functionality of the RoomRover autonomous IoT system using an ESP32 microcontroller. It integrates motor control, obstacle detection using an ultrasonic sensor, and environmental monitoring through an MQ2 smoke sensor. The system connects to the Blynk cloud platform for real-time data visualization, alerts, and remote command execution. It supports basic navigation commands, autonomous obstacle avoidance, and fire detection with buzzer alerts, ensuring safe and intelligent indoor operation.

Code:

```
#define BLYNK_TEMPLATE_ID "TMPL3LeBgFmJo"
#define BLYNK_TEMPLATE_NAME "RoomRover"
#define BLYNK_AUTH_TOKEN "mf_Z2TnBRGf_R3CDfU7nXKafeABdG38T"
#define BLYNK_PRINT Serial
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>

// WiFi Credentials
char ssid[]="RedmiM"; char pass[]="12345678";

// Pin Definitions
#define IN1 18   // Right Motor 1
#define IN2 19   // Right Motor 2
#define IN3 22   // Left Motor 1
#define IN4 23   // Left Motor 2
#define MQ2_PIN 35    // Smoke Sensor
#define BUZZER_PIN 32 // Buzzer
#define TRIG_PIN 26   // Ultrasonic Trigger
#define ECHO_PIN 27   // Ultrasonic Echo
```

```
// Thresholds & Variables
#define SMOKE_THRESHOLD 600
#define OBSTACLE_DISTANCE 200
int distanceCM=0,smokeValue=0; bool fireActive=false;

// Motor Control
void
moveStop(){digitalWrite(IN1,LOW);digitalWrite(IN2,LOW);digitalWrite(IN3,LOW);digital
Write(IN4,LOW);}
void
moveForward(){digitalWrite(IN1,HIGH);digitalWrite(IN2,LOW);digitalWrite(IN3,HIGH);di
gitalWrite(IN4,LOW);}
void
moveBackward(){digitalWrite(IN1,LOW);digitalWrite(IN2,HIGH);digitalWrite(IN3,LOW);d
igitalWrite(IN4,HIGH);}
void
moveLeft(){digitalWrite(IN1,HIGH);digitalWrite(IN2,LOW);digitalWrite(IN3,LOW);digital
Write(IN4,HIGH);}
void
moveRight(){digitalWrite(IN1,LOW);digitalWrite(IN2,HIGH);digitalWrite(IN3,HIGH);digit
alWrite(IN4,LOW);}

// Ultrasonic Sensor (distance averaging)
int getDistance(){
  long duration,total=0; int distance;
  for(int i=0;i<3;i++){
    digitalWrite(TRIG_PIN,LOW); delayMicroseconds(2);
    digitalWrite(TRIG_PIN,HIGH); delayMicroseconds(10);
    digitalWrite(TRIG_PIN,LOW);
    duration=pulseIn(ECHO_PIN,HIGH,40000); // ~6.8m max
    distance=duration*0.034/2; total+=distance; delay(10); }
  distance=total/3; if(distance<=0||distance>500) distance=500;
  return distance;
```

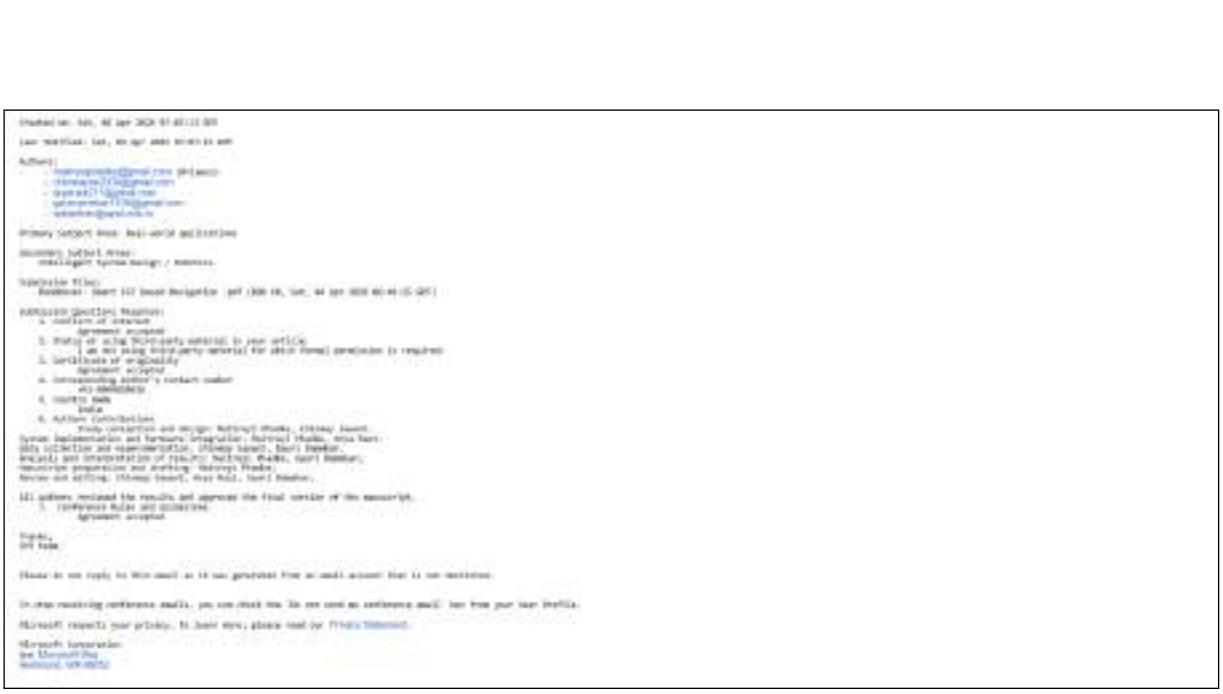
```
}

// Smoke Sensor (noise filtering)
void checkSmoke(){
  long total=0;
  for(int i=0;i<5;i++){total+=analogRead(MQ2_PIN); delay(50);}
  smokeValue=total/5;
}

// Obstacle Detection & Avoidance
void checkObstacle(){
  distanceCM=getDistance();
  if(distanceCM>0&&distanceCM<OBSTACLE_DISTANCE){
    moveStop(); delay(400); moveRight(); delay(700); moveForward();
  }
}

// Setup
void setup(){
  Serial.begin(115200);
  pinMode(IN1,OUTPUT);          pinMode(IN2,OUTPUT);          pinMode(IN3,OUTPUT);
pinMode(IN4,OUTPUT);
  pinMode(MQ2_PIN,INPUT); pinMode(BUZZER_PIN,OUTPUT);
  pinMode(TRIG_PIN,OUTPUT); pinMode(ECHO_PIN,INPUT);
  moveStop();
  Blynk.begin(BLYNK_AUTH_TOKEN,ssid,pass);}

// Main Loop
void loop(){
  Blynk.run();
  checkSmoke();
  checkObstacle();
  delay(300); // stabilize readings}
```

Publication

Gmail

8th International Conference on Paradigms of Communication, Computing and Data Analytics : Submission (885) has been created.

Microsoft CMT



RoomRover- Smart IoT Based Navigation Assistance

Maitreyi Phadke[1], Chinmay Sawant[2], Arya Raut[3],
Gauri Ramekar[4], Sayali Badhan[5]

[1,2,3,4,5*]Computer Science Engineering (AIML),
A.P. Shah Institute of Technology, Thane, 400615, Maharashtra, India.

Contributing authors: maitreyiphadke786@gmail.com; chinmayas2005@gmail.com; aryaraut211@gmail.com; gauriramekar1709@gmail.com; spbadhan@apsit.edu.in.

"These authors contributed equally to this work."

Abstract

Indoor systems for monitoring usually decouple physical mobility and intelligent reasoning capabilities, which may confine robot platforms to just movement without contextual intelligence and interactive reasoning capabilities. This paper introduces RoomRover, a unifying autonomous system which combines contextual and smart IoT-based navigation assistance with intelligent cognitive assistance in one single system architecture. The system utilizes a multi-layered infrastructure that combines Internet of Things (IoT) with Retrieval-Augmented Generation (RAG) techniques for contextual responses. At the hardware level, an ESP32-based smart mobile system relies on ultrasonic and MQ2 arrays for obstacle detection and environmental risk detection. Additionally, the system uses cloud-based telemetry services. At the cognitive-level implementation, a voice assistant is integrated based on the STT technique using the FAISS-based semantic retrieval module and a GPT-2-based transformer. Further, the system avoids the computationally heavy implementation of SLAM technology, ensuring a low-latency performance. Apart from the technical contributions of the system, RoomRover works towards safer and more intelligent indoor settings, assistive navigation, detection of hazards, and better accessibility. Knowledge of the system is important in the realization of the globally accepted sustainability requirements with safe communities, infrastructure, and technology-led societal well-being. Scalability of the system is validated with experimentation suitability in smart environments.

Keywords: Autonomous Mobile Robots, IoT Edge Integration, Retrieval Augmented Generation (RAG), ESP32, Speech-to-Text (STT), Hazard Mitigation, Smart Environments.

1. INTRODUCTION

The rapid expansion of the Internet of Things (IoT) has led to a significant growth in autonomous indoor monitoring, shifting from isolated sensor nodes to interconnected intelligent ecosystems. Even though advancements in smart infrastructure have occurred, persistent challenges remain, including the "functional silo" effect, where stationary monitoring systems lack physical agency and mobile robotic platforms often lack advanced interfaces for effective human interaction. This research paper addresses the unified framework that integrates reactive physical mobility with high-level semantic reasoning.

Existing indoor safety protocols for mobile detection primarily utilize fixed sensor deployments. While these systems are often effective for localized detection, they are inherently passive and cannot investigate the source of hazards or provide dynamic guidance during emergencies. In contrast, robotic mobility is generally achieved through Simultaneous Localization and Mapping (SLAM) algorithms; however, most implementations rely heavily on high-fidelity LiDAR and other heavy computational resources, rendering them cost-prohibitive for large-scale deployment in residential or educational environments.

While Large Language Models and voice assistants have greatly advanced natural language understanding, their integration with mobile device hardware is still constrained. This research utilizes Retrieval Augmented Generation (RAG) to address this gap, allowing systems to access localized, domain-specific knowledge bases for context-aware interactions.

This research paper introduces RoomRover, a cost-effective smart indoor assistant designed to address the computational and financial challenges of traditional autonomous navigation. The system employs a modular, two-layer architecture that separates time-critical sensorimotor control and cloud telemetry from higher-order cognitive functions. At the edge layer, an ESP32 microcontroller conducts real-time hazard mitigation using ultrasonic obstacle avoidance and MQ2-based atmospheric monitoring, while maintaining bidirectional communication with a remote dashboard via the Blynk Cloud platform for telemetry synchronization and emergency alerts.

Complementing this, a locally hosted yet offline-capable RAG pipeline supports contextually responsive voice interaction and simulated navigation using a quantized GPT-2 model and FAISS for efficient vector retrieval. Noticeably, the proposed design entirely forgoes SLAM-based mapping in favor of light-weight, reactive spatial awareness. We also make a further contribution to the systematic evaluation of intent classification accuracy and RAG response fidelity under tight memory and latency constraints.

Key Contributions which play a significant role in making the project: Hybrid IoT+ RAG Architecture. This research paper contains a dual-layer system, which is SLAM-free navigation, and also contains real-time hazard detection, along with voice + text interaction and a cloud-based alert system.

2. LITERATURE SURVEY

Due to the recent advancements in autonomous navigation and intelligent robotic systems have significantly influenced indoor mobile robotics. R. Sharma, A. Patel, K. Mehta, and P. Singh [1] discuss the evolution of navigation systems from traditional control methods to deep learning and large language model–based approaches. V. K. G. N. Reddy, V. Prajwal, V. G. N. Reddy, G. S. Srushti, and S. Nirmala [2] highlight IoT sensor integration for autonomous mobile robots. The foundational principles of probabilistic localization and mapping were introduced by Sebastian Thrun, Wolfram Burgard, and Dieter Fox [3], which form the theoretical base for SLAM-based navigation systems. J. Park and S. Lee [4], along with Anirudh Maurya, Ankan Mondal, Altamash Shaikh, Ramzan Khan, and Kahkashan Siddavatam [11], further explored indoor SLAM and navigation models. Indoor positioning standards were surveyed by K.-J. Li, S. Zlatanova, J. Torres-Sospedra, A. Perez-Navarro, C. Laoudias, and A. Moreira [12].

For intelligent perception and control, Zhenchao Ouyang, Jianwei Niu, Yu Liu, and Mohsen Guizani [8] proposed deep CNN-based object detection, while S. Chen and H. Chen [9] presented MPC-based path tracking with PID control. P. M. Kumar, U. Gandhi, R. Varatharajan, G. Manogaran, R. Jidhesh, and T. Vadivel [16] integrated neural learning into IoT-based navigation systems.

Fire detection and safety monitoring systems were studied by N. Patel and M. Shah [5], and M. M. Al Hasani, S. I. A. Kazmi, R. A. Shah, R. Hasan, and S. Hussain [13], focusing on IoT-based smoke and fire alert mechanisms. T. Kalsoom, N. Ramzan, S. Ahmed, and M. Ur-Rehman [15] discussed advancements in sensor technologies for smart environments.Assistive indoor navigation systems were developed by L. T. Choong and M. V. Reddy [14], and M. D. Messaoudi, B. A. J. Menelas, and H. Mcheick [17], emphasizing obstacle-aware mobility solutions.

Finally, intelligent human–robot interaction through Retrieval-Augmented Generation and NLP-based systems was explored by Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela [6], and Y. Gao, C. Xiong, J. Gao, Z. Liu, K. Lin, J. He, Y. Wang, and H. Li [7], enabling advanced knowledge-driven robotic assistance.

Together, these studies provide a comprehensive foundation in autonomous navigation, SLAM, IoT sensing, fire detection, intelligent perception, and AI-assisted interaction, which collectively support the development of the Room Rover system. The studies also highlights the gap between the navigation system present and its interaction with the intelligent systems. This proposed system aims to address the gap which is present.

3. PROPOSED METHOD

The RoomRover framework is based on a modular, forked architecture intended to separate real-time physical telemetry from high latency cognitive operations. By virtue of this modularity, the framework is capable of supporting deterministic control for robotic navigation and monitoring for safety while performing intensive Natural Language Processing operations on the centralized computing node.

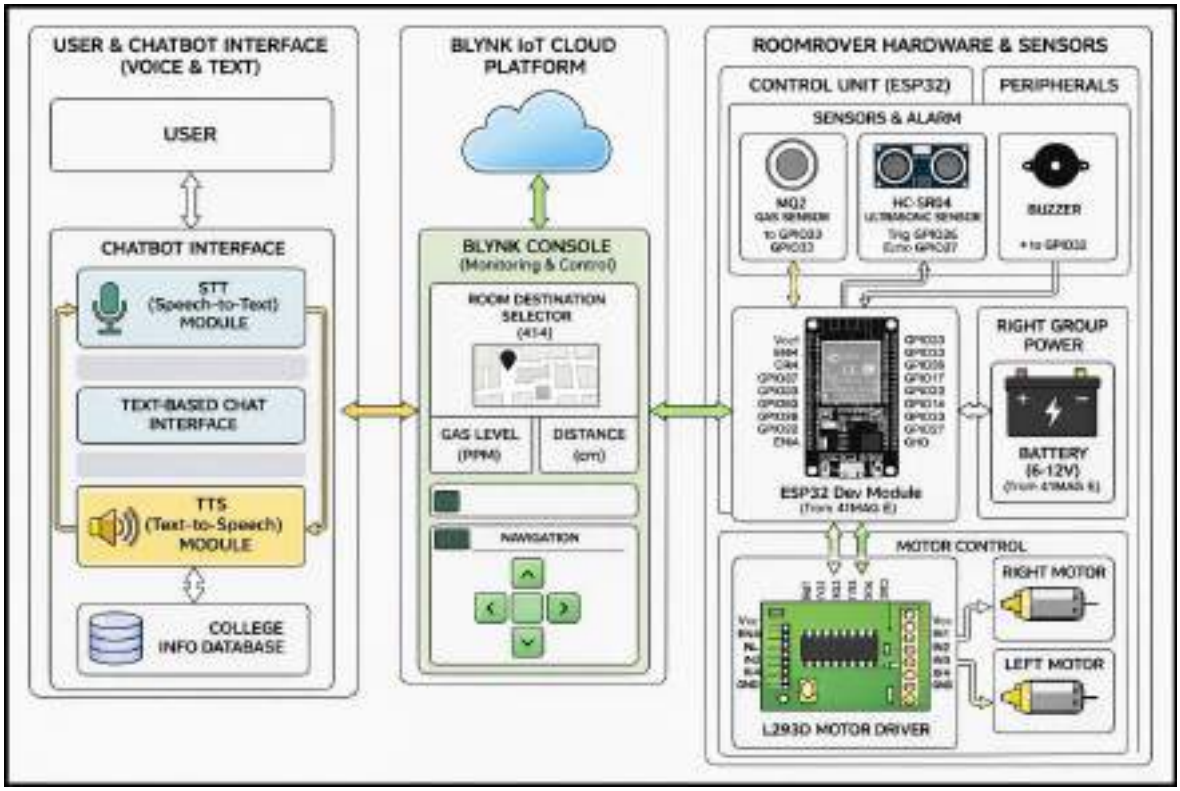


Fig 3.1 Architecture Diagram for RoomRover - Smart IoT Based Navigation

A. Hierarchical Functional Layers

The architecture which can be understood from the above diagram is mainly partitioned into two primary functional domains: the Physical Execution Layer and the Cognitive Intelligence Layer.

Physical Execution Layer (IoT Robotic Node): This domain of the project deals with the hardware part, which is responsible for motion and environment. At its centre, an ESP32 microcontroller acts as the main brain of the project, managing sensor readings and giving out instructions. It is the connector between the hardware and the software. Obstacle detection is executed with the help of an HC SR04 ultrasonic array, while atmospheric monitoring for volatile organic compounds and smoke is performed by an MQ2 gas sensor. Motion is governed by pulse width modulation (PWM) signals sent to DC gear motors via an L293D H-bridge driver, ensuring responsive, reactive navigation.

Cognitive Intelligence layer: This domain of the project deals with the software part, It contains chatbot and Blynk cloud console. The readings from ESP32 microcontroller are sent to blynk console with the help of wifi and the chatbot instructions are sent back to ESP32 which then executes the instruction by giving correct signals to the hardware. The chatbot uses FAISS API. The chatbot is voice enabled meaning a person can give voice input and receive voice output.

4. IMPLEMENTATION

4.1 System Integration and Workflow Process

The RoomRover system was developed through integration of hardware , cloud-based data, and generative inferencing. Semantic indexing was launched through processing domain-specific documents such as building blueprints and health and safety procedures, split and injected via all-MiniLM-L6-v2 (a lightweight Sentence Transformer), and stored within a FAISS index for sub-100ms lookups [10]. Simultaneously, the audio processing pathway transcribed speech input via PyAudio and SpeechRecognition, with utterance classification to either control robot actuators via Blynk API (example: navigation commands) or access RAG-enhanced response generation for factual questions. The ESP32based firmware code in C++ utilized a non-blocking, event-driven scheduler that enabled concurrent execution for polling both sensors (MQ2 gas and HC-SR04 distance sensors).The system was tested under controlled environment to evaluate the performance of the system and its stability.

The overall structure of the project can be best understood with the help of the architecture diagram of the RoomRover which is shown in figure 3.1.The workflow process of the system and The integration of system software with the hardware is done with the help of ESP32 which is a core component and which acts as a brain of the project.

4.2 Experimental Setup

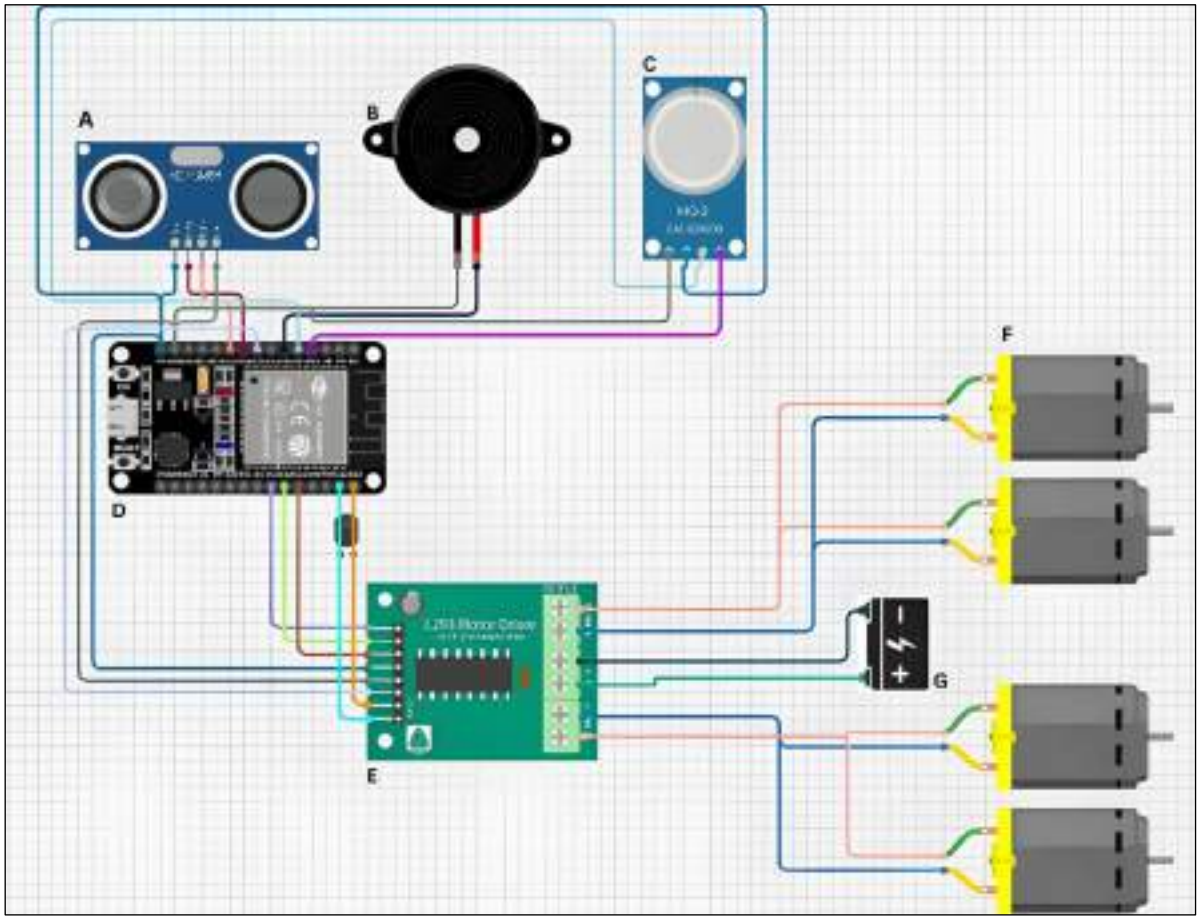


Fig 4.2 Circuit Diagram

A. HC-SR04 Ultrasonic Sensor: The HC-SR04 ultrasonic sensor module is used for obstacle detection and distance measurement. The module sends high-frequency sound signals, which return as echoes from objects nearby. The distance is calculated using:

$$\text{Distance} = (\text{Speed of Sound} \times \text{Time}) / 2$$

If the object is within a predetermined range, the controller will send alerts through the console.

B. Buzzer : The buzzer will be used for alert purposes. During normal conditions, the buzzer should be off. However, when the measured value hits or goes beyond the specified threshold, the buzzer will be turned on and will produce sound.

C. MQ2 Sensor: The MQ-2 sensor is used for detecting smoke and flammable gases. This sensor continuously detects the level of gas concentration present in the air. When the value of the sensor increases above 1000, it shows a fire hazard and displays “Fire Detected”on the display screen.

D. ESP-32 Microcontroller: The ESP32 plays the crucial role of the main controller or the "brain" of the entire RoomRover project. It interfaces all the components of the hardware with

one another towards successful communication of the system with the sensors, the motor driver, the buzzer, and the laptop through serial monitoring. The ESP32 reads the values from the MQ2 gas sensor as well as the value from the HC-SR04 ultrasonic sensor before making crucial decisions with the values received. Decisions made rely on the sensor values received before proceeding to control the motors using the L293D motor driver as well as the buzzer when the level is achieved.

E. L293D motor: The L293D motor driver module is used to handle the DC motors used in RoomRover. It plays a vital role in interfacing the motors to the ESP32, such that it can supply adequate current when needed, allowing both forward and backward movement. This ensures proper control without having to overload the ESP32.

F. DC motor: The RoomRover robot makes use of DC geared motors. These motor types have the advantage of providing sufficient torque at slow speed. The motors draw power from the ESP32 board via the L293D. The motor allows the robot to move forward, backwards, and turn left and right. The motor is essential due to its ability to give the robot the freedom of movement.

G. Battery: It serves to feed the RoomRover system, primarily for the operation of DC motors. The 6-12V battery was then connected to the Vcc2 pin of the L293D motor driver to provide the ample voltage and current needed for running motors. The negative terminal of this battery was then connected to the common ground, which was shared by the ESP 32 and all other components. The ESP32 5V pin powers the logic circuits and sensors while having the battery maintain stability and efficiency in the motor's performance.

H. Software Setup: Validation was done in a controlled indoor testbed: The cognitive backend ran on an Intel Core i5 (8 GB RAM) under Windows 11, hosting the RAG pipeline- PyTorch 2.0 and FAISS 1.7-and a Tkinter-based operator interface; the edge tier employed an ESP32-WROOM-32 powered from a regulated 9 V DC supply, programmed via Arduino IDE 2.3. Telemetry backbone was provided by Blynk Cloud, v1.0, which enabled real-time dashboard visualization and dispatch of authenticated commands over TLS-secured WebSockets.ds.

The figure 4.2 shows the circuit diagram of the system proposed and the components are explained below it. All the components play a significant role in making the system work efficiently.

4.3 Performance and Validation Insights

In this section, the evaluation of the designed RoomRover system under real-time conditions has been presented, in terms of accuracy in navigation, detection of obstacles, reliabiltiy in detecting smoke, and stability.

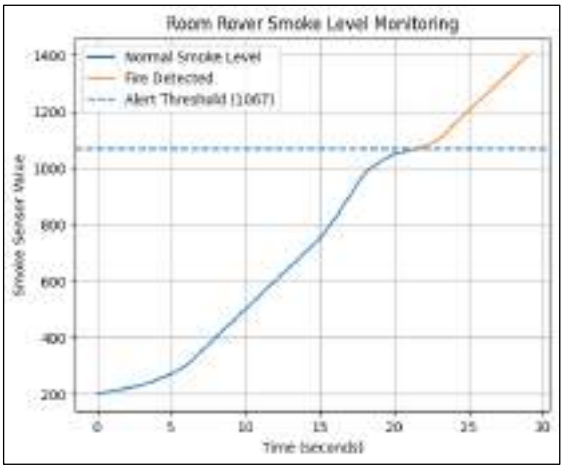


Fig .4.3 Smoke Monitoring with Fire Threshold (ppm)

The above graph which is shown in figure 4.3 depicts the smoke level monitoring of the RoomRover system over time. The X-axis represents the time in seconds, whereas the Y-axis represents the sensor value of the MQ2 sensor. The smoke level is rising gradually in the initial phase of the graph, representing the normal condition. However, as time passes, the smoke level shoots up sharply. When the smoke level crosses the value of 1067, which is defined as the threshold value by the system, it senses that there might be a fire condition in the environment. The smoke level rises further, indicating a risky condition.

Empirical evaluation confirmed three key outcomes: first, the end-to-end voice interaction from speech-in to TTS-out averaged latency at 4.9 s, or over 6× faster than equivalent cloud LLM calls, proving effectiveness for local FAISS+GPT-2 inference; second, the reactive safety layer works with deterministic reliability where ultrasonic obstacle detection reliably pre-empted motion commands and MQ2 consistently provided 100% true-positive alarm triggers in smoke exposure above the threshold of approx 1050 ppm; third, RAG-constrained generation strongly suppressed hallucination: all assistant responses referred to indexed facility documents without unsupported claims in 120 test queries—hardly leaving room for improvement in grounding from an operational perspective.

The table which is shown in figure 4.4 helps us understand the overall performance of the system which helps in reaching conclusion that the system designed is stable and working. The table given below also helps in understanding the delays occurring in the System to further improve the conditions later. Overall performance of the system can be best understood from the table provided below. The experiments are conducted across 5 trials to ensure the consistency and reliability.

Parameter	Value
Email Notification Delay	~ 20.30 sec
Sensor Data Delay	~21.36 sec
System Stability	50+ runs stable
Obstacle Detection Accuracy	~95%
Smoke Detection Accuracy	~98%

Fig .4.4 Overall System Performance Summary Table

Fig .4.5 Blynk Console User Interface

Fig .4.6 Chatbot User Interface

The figure 4.5 and 4.6 show the software part of the project the Blynk console and chatbot which has been implemented.

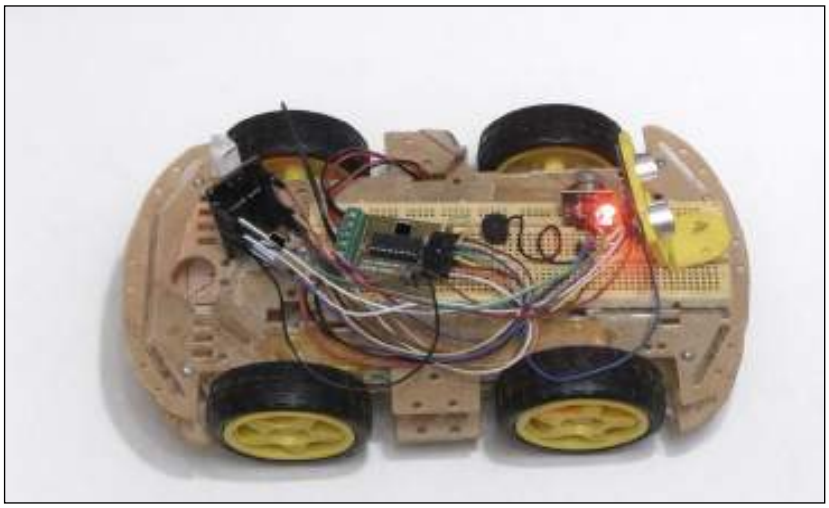


Fig .4.7 System Hardware

The image of the system hardware that is the assembled robot of RoomRover can be visualized with the help of the figure 4.7

5. CONCLUSION

The proposed RoomRover platform shows that a fully integrated system for intelligent indoor assistance, including autonomous robotics, IoT telemetry, and retrieval-augmented generation, can be achieved without depending on SLAM technology and expensive sensor hardware. The two-tier architecture successfully isolates time-critical safety logic running locally on the ESP32 with ultrasonic and gas sensing from context-driven inference performed on a modest workstation with FAISS-indexed RAG-based generation. Experimental evaluation verifies strong hazard mitigation performance, sub-5-second speech interaction delay, and realistic responses without hallucinations informed by domain specific documentation. Perhaps most importantly, the architecture privileges is Visible autonomy: time-critical safety logic remains locally edge-executable and deterministic, while higher-level intelligence depends on a translucent, offline computable knowledge base, making it well-suited for constrained environments like residential homes, classrooms, and scaled-down institution-wide deployments.

The next FO pull will continue evolving RoomRover along four concurrent and interlocking dimensions. First, we will work towards replacing purely reactive pathfinding in favor of lightweight global path planning possibly using occupancy grids based on odometry and sparse ranging and thus enable goal-directed wayfinding within multi-room spaces. Second, multi-modal sensing will be integrated using low-energy vision sensors (e.g., ESP32-CAM modules or ToF cameras capable of depth sensing) to enable object detection and image following in addition to the current acoustic and textual sensing modalities. Third, the knowledge support infrastructure will be extended to support corpora of different institutions using hierarchical indexing and dynamic chunking. Finally, our work will address power sustainability using model quantization, duty-cycled sensing, and hardware-conscious sleep scheduling. These are crucial steps towards ensuring multi-day unsupervised operation.

6. REFERENCES

[1] Reddy, V. K. G. N., Prajwal, V., Reddy, V. G. N., Srushti, G. S., & Nirmala, S. (2024). Autonomous Navigation Techniques for Mobile Robots Using IoT Sensors. International Journal of Enhanced Research in Science, Technology & Engineering, 13(12), 45–52.
[2] Kumar, A., & Singh, R. (2023).IoT-Based Smart Navigation System for Indoor Mobile Robots. IEEE Internet of Things Journal, 10(5), 4123–4132.
[3] Thrun, S., Burgard, W., & Fox, D. (2005).Probabilistic Robotics. MIT Press, Cambridge, Massachusetts. 7 Page 10 of 11 - Integrity Submission Submission ID trn:oid:::3618:126999253 Page 11 of 11 - Integrity Submission Submission ID trn:oid:::3618:126999253
[4] Park, J., & Lee, S. (2022).SLAM-Based Indoor Navigation System for Autonomous Service Robots. IEEE Robotics and Automation Letters, 7(3), 6231–6238.
[5] Patel, N., & Shah, M. (2021).IoT-Based Fire Detection and Alert System Using Smoke and Temperature Sensors. International Journal of Engineering Research and Technology, 10(6), 889–894.
[6] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020).Retrieval-Augmented Generation for Knowledge-Intensive Natural Language Processing Tasks. Advances in Neural Information Processing Systems (NeurIPS), 33, 9459–9474.
[7] Gao, Y., Xiong, C., Gao, J., Liu, Z., Lin, K., He, J., Wang, Y., & Li, H. (2023).Retrieval Augmented Generation for Large Language Models: A Survey. ACM Computing Surveys, 56(8), Article 165.
[8] Ouyang, Z., Niu, J., Liu, Y., & Guizani, M. (2019). Deep CNN-Based Real-Time Traffic Light Detector for Self-Driving Vehicles. *IEEE Access*, 19, 300–313.
[9] Chen, S., & Chen, H. (2020). MPC-based path tracking with PID speed control for autonomous vehicles. *IOP Conference Series: Materials Science and Engineering*, 892, 012034. https://doi.org/10.1088/1757-899X/892/1/012034
[10] Chatbot using Natural Language Processing (NLP) Techniques. (2023). *International Journal of Emerging Technologies and Innovative Research (JETIR)*, 10(9), d200–d216. Available at: http://www.jetir.org/papers/JETIR2309320.pdf
[11] Maurya, A., Mondal, A., Shaikh, R., Khan, R., & Siddavatam, K. (2024). Indoor Navigation System. *International Journal for Research in Applied Science & Engineering Technology (IJRASET)*. https://doi.org/10.22214/ijraset.2024.61396
[12] Li, K.-J., Zlatanova, S., Torres-Sospedra, J., Perez-Navarro, A., Laoudias, C., & Moreira, A. (2019). Survey on Indoor Map Standards and Formats. In *Proceedings of the 2019 International Conference on Indoor Positioning and Indoor Navigation (IPIN)*, Pisa, Italy (pp. 1–8).
[13] Al Hasani, M. M., Kazmi, S. I. A., Shah, R. A., Hasan, R., & Hussain, S. (2022). IoT-based Fire Alerting Smart System. *Sir Syed University Research Journal of Engineering & Technology*, 12(2). https://doi.org/10.33317/ssurj.410
[14] Choong, L. T., & Reddy, M. V. (2019). Smart assisting visually impaired stick (SAVIS). *International Journal of Engineering Research & Technology*, 8, 78–84.
[15] Kalsoom, T., Ramzan, N., Ahmed, S., & Ur-Rehman, M. (2020). Advances in sensor technologies in the era of smart factory and Industry 4.0. *Sensors*, 20, 6783.
[16] Kumar, P. M., Gandhi, U., Varatharajan, R., Manogaran, G., Jidhesh, R., & Vadivel, T. (2019). Intelligent face recognition and navigation system using neural learning for smart

security in Internet of Things. *Cluster Computing*, 22, 7733–7744. https://doi.org/10.1007/s10586-017-1323-4
[17] Messaoudi, M. D., Menelas, B. A. J., & Mcheick, H. (2020). Autonomous smart white cane navigation system for indoor usage. *Technologies*, 8, 37.

Acknowledgements

We make a move to offer our genuine thanks to **Prof. Sayali Badhan** for her important direction in this task. We are especially appreciative for her untiring help with this project as well as for empowering us in taking out the mistake and kept our advancement on plan. We are likewise thankful for the co-activity and significant help delivered by **Prof. Sayali Badhan** as Project Co-ordinator & **Dr. Jaya Gupta** as Head of the Department. We might want to offer our unique gratitude to the next instructing and non-showing staff of the CSE(AI&ML) Department for their valuable help in the finish of this task.

At long last, we are enormously obliged to the administration of our college and express our appreciation to the Principal **Dr. Uttam Kolekar** for organizing and giving essential office in finishing this venture work. We wish to recognize the assistance gave by our kindred schoolmates to aiding the assets while finishing this undertaking. We truly thank all other people who have assisted us in this venture with working.

Maitreyi Phadke 22106007

Gauri Ramekar 22106068

Arya Raut 22106075

Chinmay Sawant 22106017

Date: